

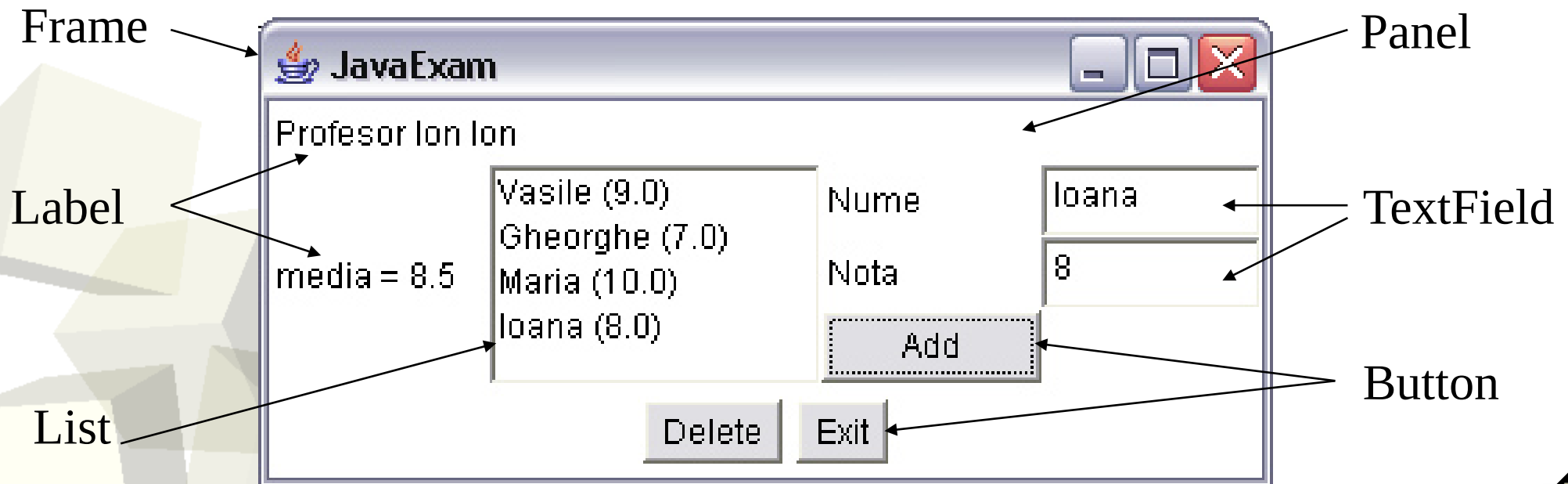


- 1. Programarea condusa de evenimente**
- 2. Model-View-Controller**
- 3. Ierarhia AWT**
- 4. Proiectarea interfetei**
- 5. Crearea componentelor**
- 6. Alegerea containerelor**
- 7. Alegerea layout managerelor**
- 8. Afisarea interfetei**
- 9. Tratarea evenimentelor**

- Realizarea unei interfete grafice este o problema dificila, de aceea se folosesc componente grafice “prefabricate”
- O interfata grafica trebuie sa fie intuitiva, usor de folosit
- Atunci cand se realizeaza o interfata grafica, **initiativa actiunilor** apartine utilizatorului (apasarea unui buton, selectarea unui check-box, etc.)

- Orice interactiune a utilizatorului cu interfata grafica se numeste **eveniment**
- **Programarea condusa de evenimente** este stilul de programare in care ne propunem ca programul sa reactioneze la anumite evenimente (provocate de utilizator, sau de alte conditii)

- Realizarea unei interfete se face in doi pasi:
 - realizarea interfetei grafice
 - descrierea actiunilor ce trebuie realizate pentru fiecare eveniment posibil

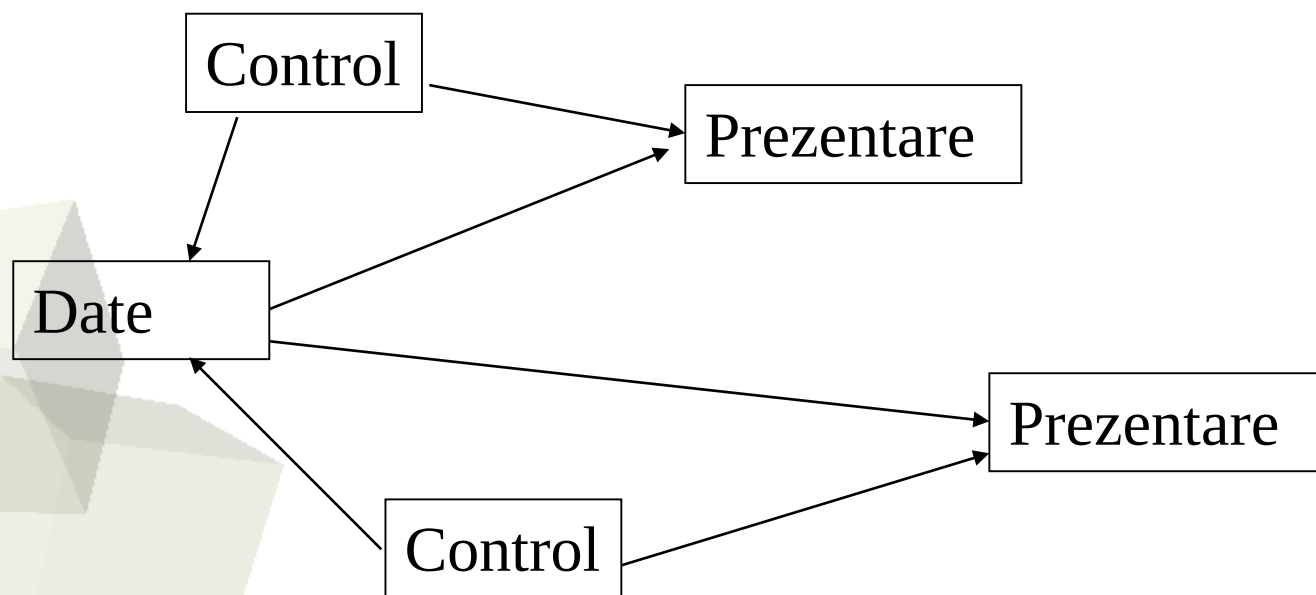




1. Programarea condusa de evenimente
2. **Model-View-Controller**
3. Ierarhia AWT
4. Proiectarea interfetei
5. Crearea componentelor
6. Alegerea containerelor
7. Alegerea layout managerelor
8. Afisarea interfetei
9. Tratarea evenimentelor



- In proiectarea unei interfete grafice trebuie sa tinem cont de trei elemente:
 - date
 - prezentarea grafica a datelor
 - controlul datelor (actiunile efectuate la aparitia evenimentelor)

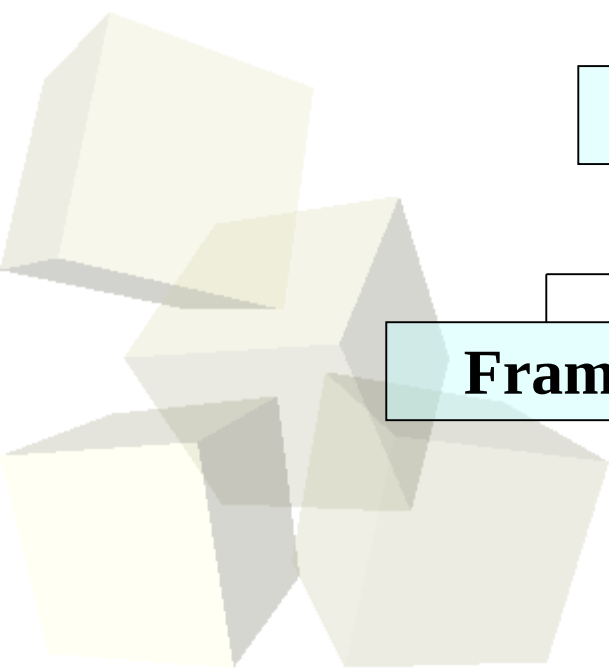
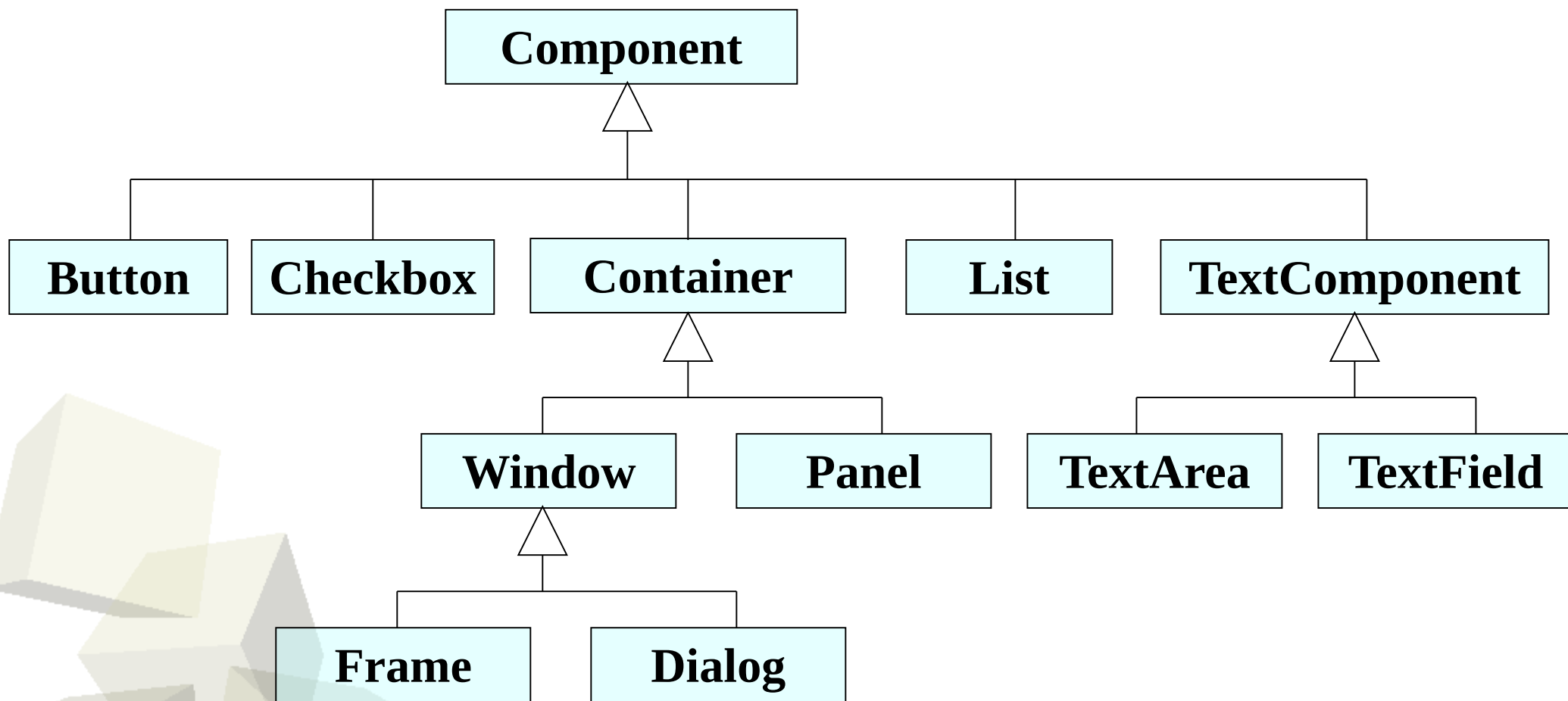




- Acest lucru se face respectand sablonul MVC (Model-View-Controller) care separa cele trei elemente in:
 - **Model**: clasele care tin datele propriu zise (ex. clasa Student)
 - **View**: clasele care prezinta datele din Model in diverse formate grafice (ex. o lista de studenti)
 - **Controller**: clasele care descriu actiunile asupra datelor la interactionarea utilizatorului cu interfata grafica (ex. ce se intampla cand selectam un student)



1. Programarea condusa de evenimente
2. Model-View-Controller
3. **Ierarhia AWT**
4. Proiectarea interfetei
5. Crearea componentelor
6. Alegerea containerelor
7. Alegerea layout managerelor
8. Afisarea interfetei
9. Tratarea evenimentelor





- Cea mai simpla modalitate de a implementa interfețe grafice este oferita in Java prin **AWT** (Abstract Window Toolkit)
- Clasa **Component** este parinte pentru orice componenta grafica (butoane, liste, checkbox, zone de text)
- Clasa **Container** este parinte pentru orice fereastră ce poate contine alte componente



- Din clasa **Container** sunt derivate doua clase:
 - **Window** – este o fereastră de sine statatoare si poate contine alte componente si panel-uri
 - **Frame** – este fereastră aplicatiei
 - **Dialog** – este o mica fereastră de interogare sau de informare
 - **Panel** – nu este o fereastră de sine statatoare, ci trebuie continuta intr-o alta fereastră de tip Frame, Dialog sau Panel



1. Programarea condusa de evenimente
2. Model-View-Controller
3. Ierarhia AWT
4. **Proiectarea interfetei**
5. Crearea componentelor
6. Alegerea containerelor
7. Alegerea layout managerelor
8. Afisarea interfetei
9. Tratarea evenimentelor



- Primul pas este proiectarea (pe hartie, sau cu un program specializat a interfetei) si are ca scop identificarea tuturor componentelor grafice

The screenshot shows a Java application window titled "JavaExam". Inside the window, there is a label "Profesor Ion Ion" and a text area displaying "media = 8.5". To the right of the text area is a list of student names and their scores: "Vasile (9.0)", "Gheorghe (7.0)", "Maria (10.0)", and "Ioana (8.0)". To the right of this list is a form with two input fields: "Nume" (Name) with the value "Ioana" and "Nota" (Grade) with the value "8". Below the "Nota" field is an "Add" button. At the bottom of the window are two buttons: "Delete" and "Exit".

Nume	Nota
Vasile	9.0
Gheorghe	7.0
Maria	10.0
Ioana	8.0



1. Programarea condusa de evenimente
2. Model-View-Controller
3. Ierarhia AWT
4. Proiectarea interfetei
5. **Crearea componentelor**
6. Alegerea containerelor
7. Alegerea layout managerelor
8. Afisarea interfetei
9. Tratarea evenimentelor



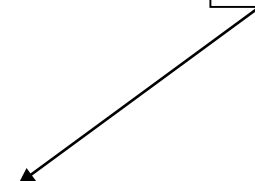
- Respectand tiparul **MVC**, vom crea o alta clasa pentru prezentarea grafica, numita `ExamenFrame`
- In constructorul acestei clase, sau intr-o functie apelata din constructor, se va construi interfata grafica
- Urmatorul pas in realizarea interfetei grafice este crearea componentelor care au fost identificate la primul pas



Crearea componentelor

```
public class ExamenFrame extends  
    WindowAdapter implements  
    ActionListener {  
    private Examen exam;  
    private Label prof;  
    private List studenti;  
    private Label media;  
    private TextField nume;  
    private TextField nota;  
    private Label lNume;  
    private Label lNota;  
    private Button exit;  
    private Button del;  
    private Button add;
```

Atribute





Crearea componentelor

```
public ExamenFrame(Examen exam) {  
    this.exam = exam;  
  
    prof = new Label(exam.getProfesor());  
    studenti = new List(5);  
    Iterator i = exam.getStudenti();  
    while (i.hasNext()) {  
        Student s = (Student) i.next();  
        studenti.add(s.toString());  
    }  
    media = new Label("media="+exam.getMedia());  
    nume = new TextField();  
    nota = new TextField();  
    lNume = new Label("Nume");  
    lNota = new Label("Nota");  
    exit = new Button("Exit");  
    del = new Button("Delete");  
    add = new Button("Add");  
}
```

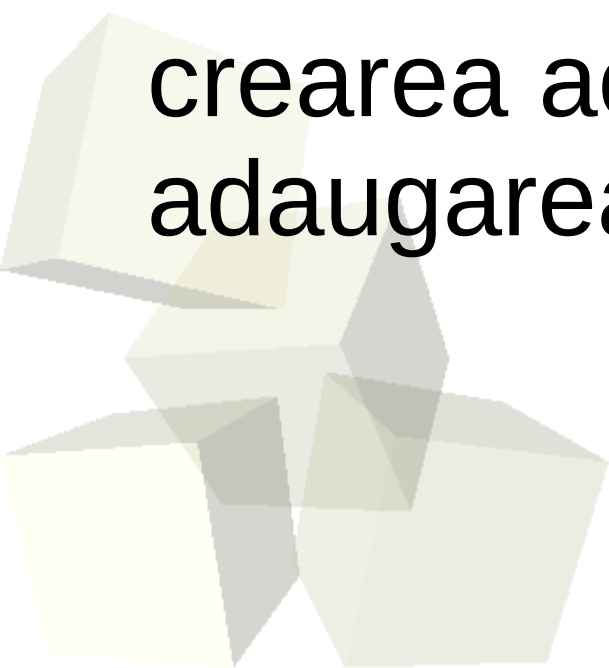


1. Programarea condusa de evenimente
2. Model-View-Controller
3. Ierarhia AWT
4. Proiectarea interfetei
5. Crearea componentelor
6. **Alegerea containerelor**
7. Alegerea layout managerelor
8. Afisarea interfetei
9. Tratarea evenimentelor



Alegerea containerelor

- Fiecare componenta grafica pe care am creat-o la pasul precedent trebuie sa fie continuta intr-un container (Frame, Panel, Dialog)
- Pasul urmator consta in alegerea si crearea acestor containere si apoi adaugarea componentelor la containere





Alegerea containerelor

...

```
Panel text = new Panel();
```

```
Panel buttons = new Panel();
```

```
Frame f = new Frame("JavaExam");
```

```
text.setLayout(new GridLayout(3, 2));
```

```
f.add(prof, "North");
```

```
f.add(studenti, "Center");
```

```
f.add(media, "West");
```

...



Alegerea containerelor

```
...  
text.add( lNume);  
text.add( nume);  
text.add( lNota);  
text.add( nota);  
text.add( add);  
  
f.add( text, "East");  
  
buttons.add( del);  
buttons.add( exit);  
  
f.add( buttons, "South");  
...
```



1. Programarea condusa de evenimente
2. Model-View-Controller
3. Ierarhia AWT
4. Proiectarea interfetei
5. Crearea componentelor
6. Alegerea containerelor
7. **Alegerea layout managerelor**
8. Afisarea interfetei
9. Tratarea evenimentelor



Alegerea layout managerelor

- Orice container are asociat un `LayoutManager` care este responsabil de aranjarea componentelor in container
 - **FlowLayout** – aranjeaza componentele in ordine de la stanga la dreapta
 - **GridLayout** – aranjeaza componentele intr-o grila
 - **CardLayout** – aranjeaza componentele una peste alta
 - **BorderLayout** – imparte containerul in cinci zone: Centru, Nord, Sud, Est, Vest



Alegerea layout managerelor

- Schimbarea layout managerului se face prin metoda

setLayout (LayoutManager l)

- **Frame** are implicit BorderLayout

- **Panel** are implicit FlowLayout



- 1. Programarea condusa de evenimente**
- 2. Model-View-Controller**
- 3. Ierarhia AWT**
- 4. Proiectarea interfetei**
- 5. Crearea componentelor**
- 6. Alegerea containerelor**
- 7. Alegerea layout managerelor**
- 8. Afisarea interfetei**
- 9. Tratarea evenimentelor**



- Dupa ce am creat interfata trebuie sa o afisam
- Acest lucru se realizeaza prin metoda **setVisible(true)**

...

```
f.pack();  
f.setVisible(true);  
}
```



- 1. Programarea condusa de evenimente**
- 2. Model-View-Controller**
- 3. Ierarhia AWT**
- 4. Proiectarea interfetei**
- 5. Crearea componentelor**
- 6. Alegerea containerelor**
- 7. Alegerea layout managerelor**
- 8. Afisarea interfetei**
- 9. Tratarea evenimentelor**



- Ultimul pas ramane scrierea codului pentru a raspunde la actiunile utilizatorului
- Pentru a raspunde la evenimentele provocate de utilizator, o clasa de tip **Controller** trebuie sa “**asculte**” dupa acele evenimente
- Orice componenta grafica permite ca mai multe clase de tip Controller sa se inregistreze la ele ca **Listener**



- In momentul aparitiei unui eveniment intr-o componenta, acea componenta (buton, checkbox, frame, etc.) va anunta toate obiectele inregistrate la ea ca **Listener**
- Pentru fiecare tip de eveniment (legate de fereastră, de butoane, de tastatura, de mouse) exista o interfata de tip Listener ce trebuie implementata de obiectul care se inregistreaza (ex. ActionListener, WindowListener)



- Interfetele de tip Listener care au multe metode, au si o implementare vida numita adaptor (WindowAdapter, etc.)
- De multe ori este mai comod sa extindem **WindowAdapter** si sa modificam doar metoda care ne intereseaza, decat sa implementam **WindowListener** si sa fim obligati sa scriem toate metodele, din care folosim doar foarte putine



Tratarea evenimentelor

```
public class ExamenFrame extends  
    WindowAdapter implements  
    ActionListener {
```

```
    ...
```

```
        f.addWindowListener(this);  
        add.addActionListener(this);  
        exit.addActionListener(this);  
        del.addActionListener(this);
```

```
        f.pack();  
        f.setVisible(true);
```

```
    }
```



- Implementarea unor interfete de tip Listener ne obliga sa implementam metodele pentru tratarea evenimentelor
 - void actionPerformed() – pentru tratarea evenimentelor de tip apasarea unui buton
 - void windowClosing() – pentru evenimente de tip inchiderea ferestrei





Tratarea evenimentelor

```
public void windowClosing(WindowEvent e) {  
    System.exit(0);  
}  
  
public void actionPerformed(ActionEvent e) {  
    if (exit == e.getSource()) {  
        System.exit(0);  
    }  
    if (del == e.getSource()) {  
        int i = studenti.getSelectedIndex();  
        if ((i >= 0) && (i < studenti.getItemCount())) {  
            studenti.remove(i);  
            exam.remove(i);  
            media.setText("media=" + exam.getMedia());  
        }  
    }  
}
```

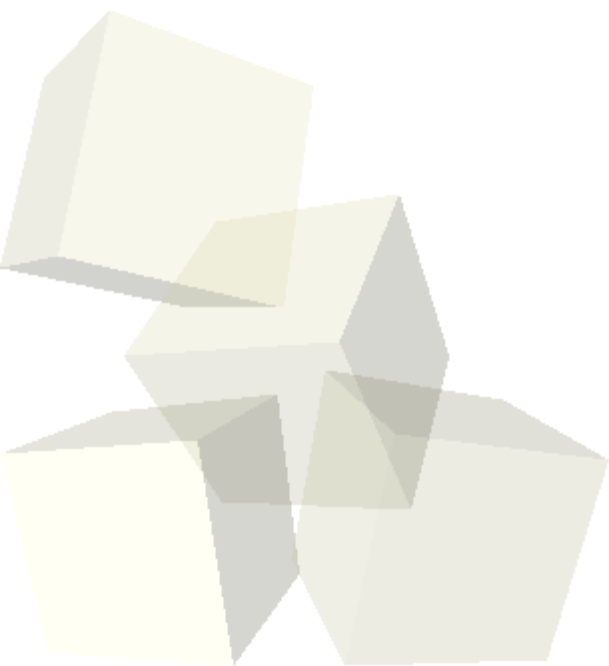


Tratarea evenimentelor

```
if (add == e.getSource()) {  
    try {  
        Student s = new Student(ume.getText());  
        s.setNota(Double.parseDouble(  
nota.getText()));  
        exam.addStudent(s);  
        student1.add(s.toString());  
        media.setText("media="+exam.getMedia());  
    } catch (StudentException ex) {  
        System.out.println("nota gresita");  
    }  
}  
}  
}
```



Interfețe utilizator cu Swing

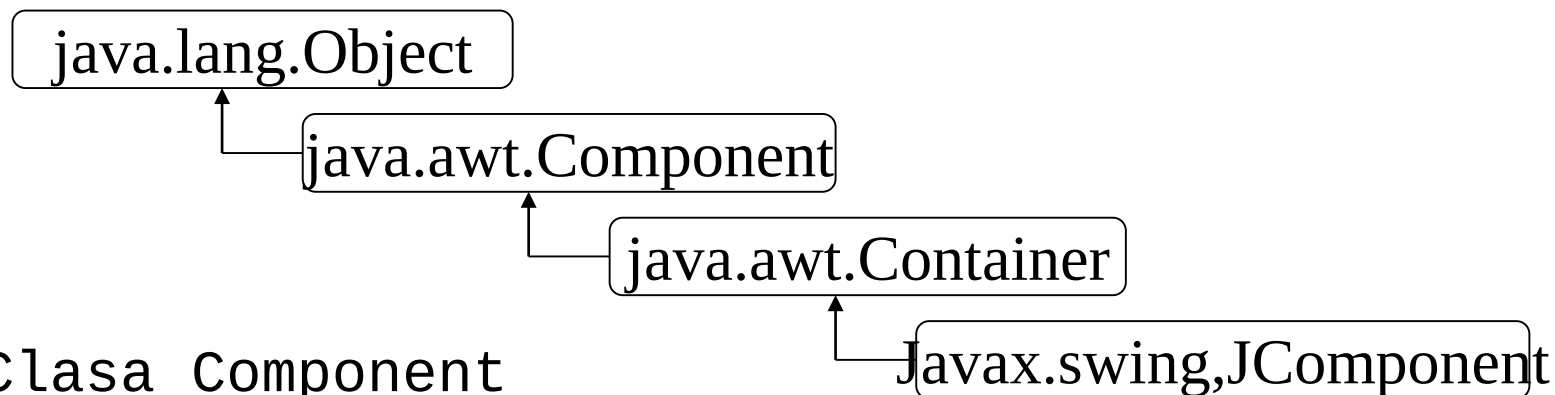




- Clase din pachetul `javax.swing`
- Câteva componente de bază
 - `JLabel`
 - `TextField`
 - `CheckBox`
 - `ComboBox`
 - `JList`
 - `JPanel`
- Cele mai multe dintre componentele swing sunt scrise complet în Java, astfel încât ele oferă portabilitate maximă și flexibilitate mai mare decât a celor din pachetul awt
 - Componentele awt sunt dependente de platformă
 - Doar anumite componente swing sunt dependente de platformă. ex, `JFrame`



- Superclase comune mai multor componente Swing



- Clasa Component

- Operații comune majorității componentelor GUI sunt în clasa Component

- Container class

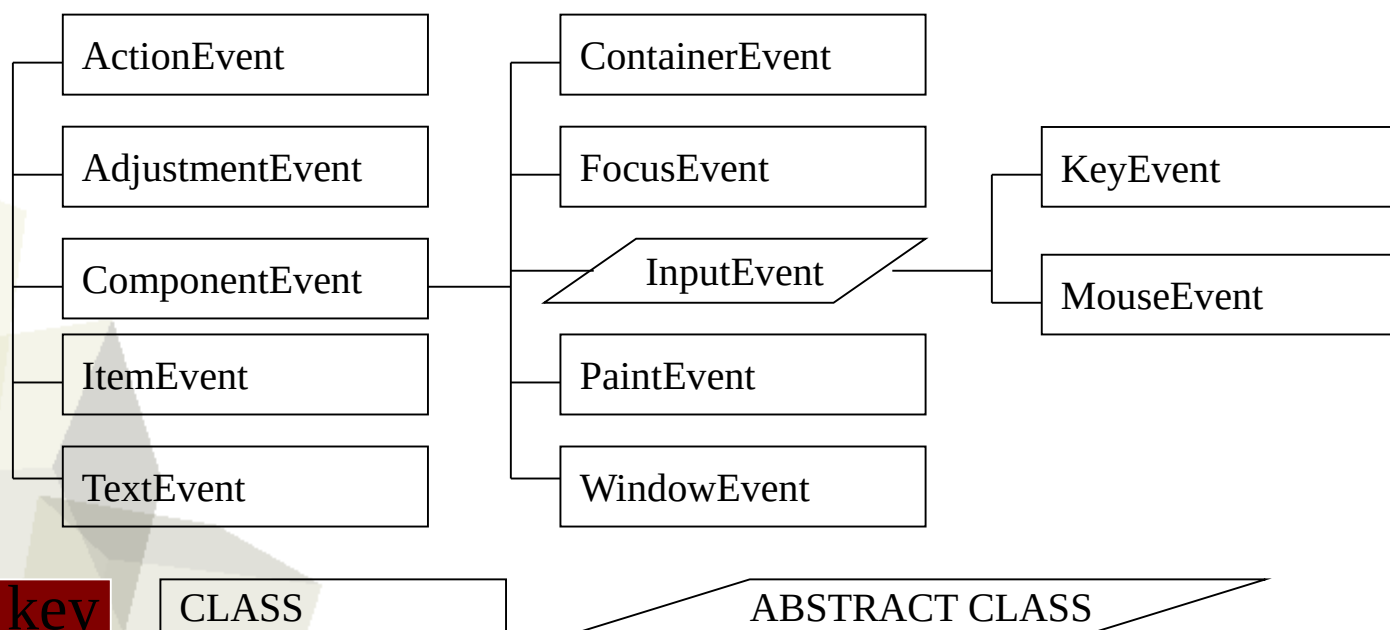
- Are două metode importante
 - `add` — adaugă componenta la un container.
 - `setLayout` — stabilește modul de poziționare a componentelor



Event-Handling Model (1)

- Unele componente GUI sunt componente *event driven* — generând evenimente
 - ex, mișcarea mouse-ului, click pe un button, tastare text în text field, selectare item de meniu, etc.
 - La interacțiunea cu utilizatorul se transmite un eveniment programului. Tipuri de evenimente sunt în `java.awt.event` and `javax.swing.event`

Exemple din `java.awt.event`

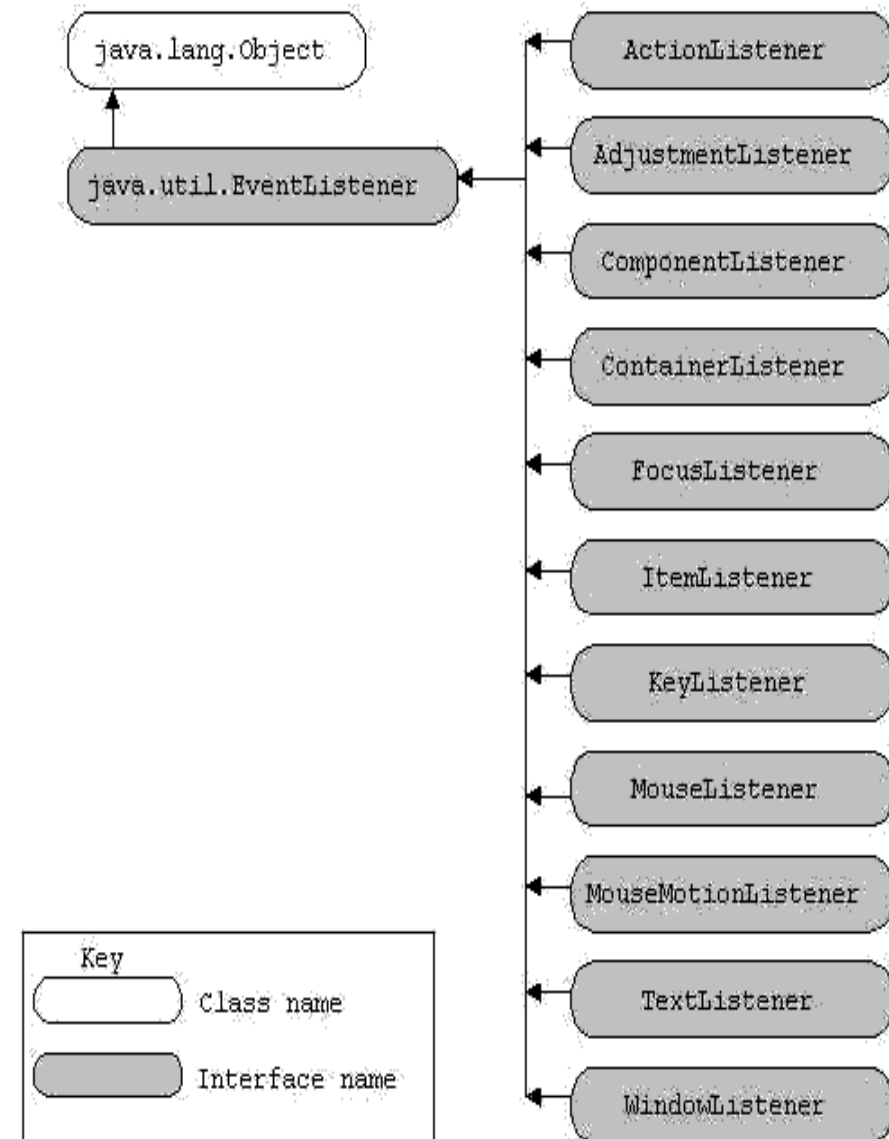




Event-Handling Model (2)

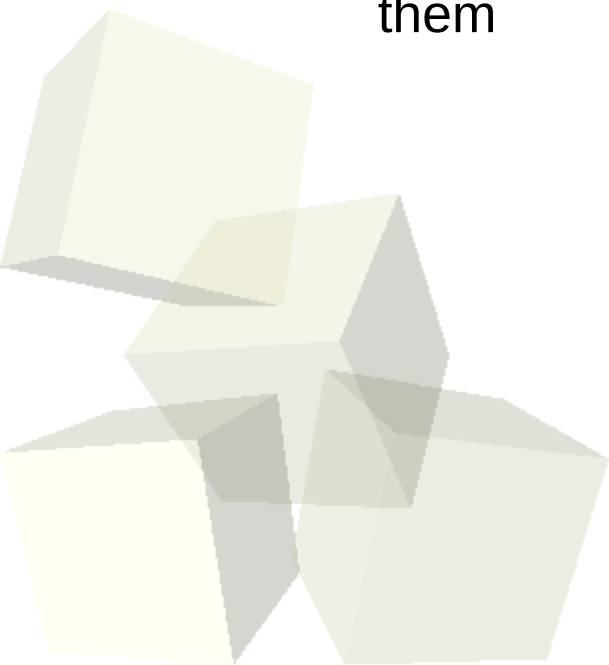
- Componentele mecanismului event-handling
 - *event source*: Sursa evenimentului, comp GUI cu care interacționează utilizatorul
 - *event object*: informația încapsulată referitoare la evenimentul produs
 - *event listener*: un obiect care este notificat de către sursa evenimentului și care furnizează răspunsul la eveniment
- Programatorul are de făcut două lucruri pentru a răspunde la eveniment
 1. Înregistrarea unui *event listener*
 - Un obiect a unei clase care implementează unul sau mai multe interfețe event-listener din `java.awt.event` și `javax.swing.event`
 2. implementarea unei metode *event handling*

Event-listener interface of package `java.awt.event`





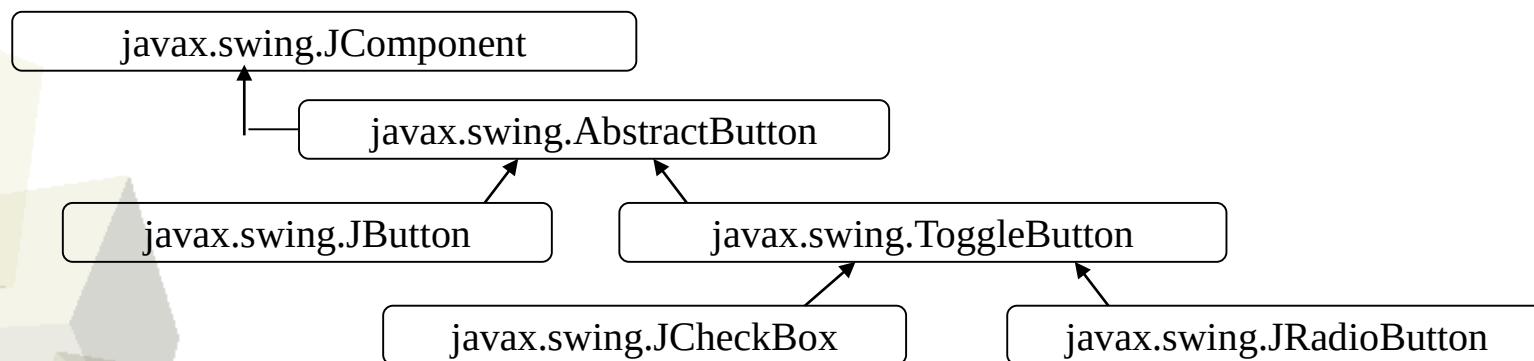
- Pentru a crea o interfață grafică interactivă este nevoie de:
 - Components
 - buttons, windows, menus, etc.
 - Events
 - mouse clicked, window closed, button clicked, etc.
 - Event listeners (interfaces) and event handlers (methods)
 - listen for events to be triggered, and then perform actions to handle them





Buttons

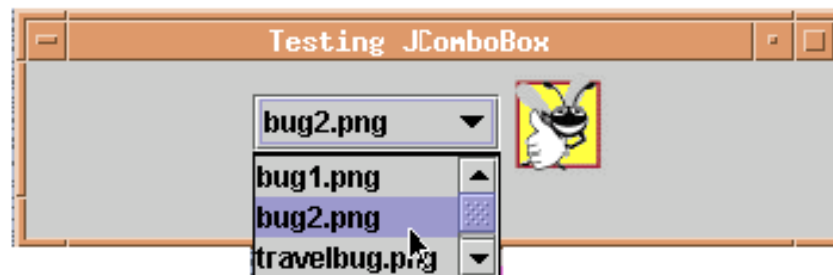
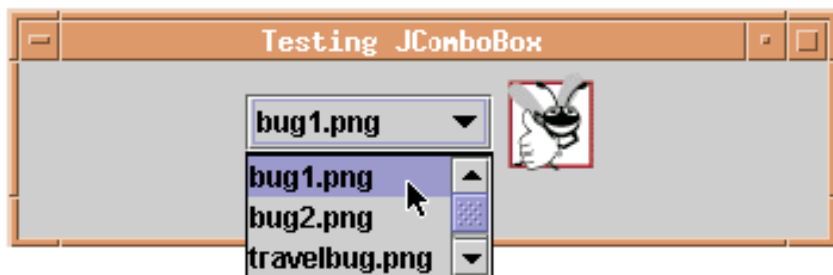
- *Button*
- Sunt câteva clase toate subclase ale clasei `AbstractButton`
 - *command buttons*: `JButton`. Generează `ActionEvent`
 - *toggle buttons*: haveau on/off sau true/false values
 - *check boxes*: un group de buttons. Generează `ItemEvent`
 - *radio buttons*: un group de buttons din care doar unul poate fi selectat. Generează `ItemEvent`



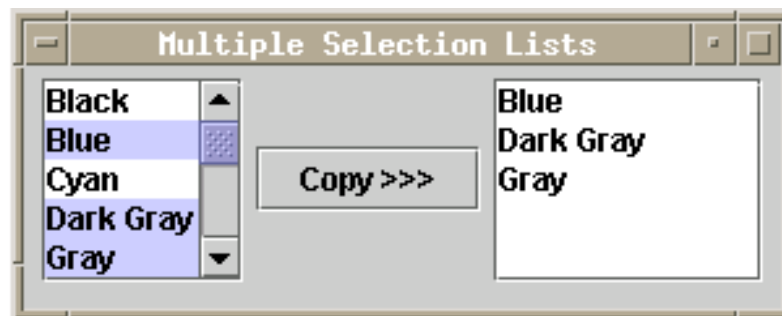
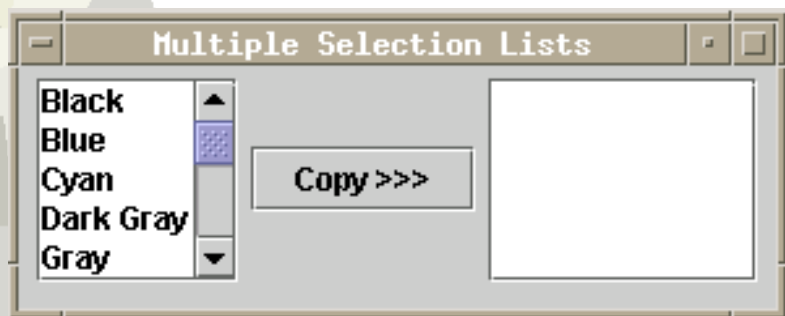


Alte componente ...

- JComboBox: O listă ascunsă de itemuri din care se poate selecta doar unul. Generează ItemEvent



- JList: O listă ce suportă atât *single selection* cât și *multiple-selection*. Generează ListSelectionEvent





- A visual Index to the Swing Components

<http://java.sun.com/docs/books/tutorial/uiswing/components/components.html>

- **Creating a GUI with JFC/Swing**

<http://java.sun.com/docs/books/tutorial/uiswing/index.html>

- **Building a User Interface**

<http://java.sun.com/developer/onlineTraining/new2java/divelog/part1/>

<http://java.sun.com/developer/onlineTraining/new2java/divelog/part2/index.jsp>

<http://java.sun.com/developer/onlineTraining/new2java/divelog/part3/>

<http://java.sun.com/developer/onlineTraining/new2java/divelog/part4/>

<http://java.sun.com/developer/onlineTraining/new2java/divelog/part5/>





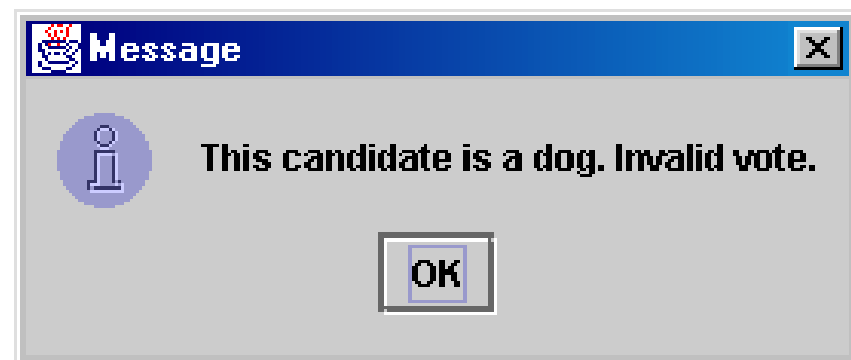
O componentă option este o fereastră dialog box pentru input/output în mod grafic

■ **avantaje:**

- simple
- flexibile
- Aspectul

■ **dezavantaje:**

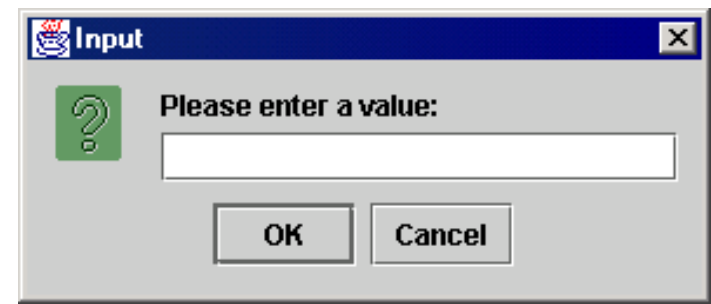
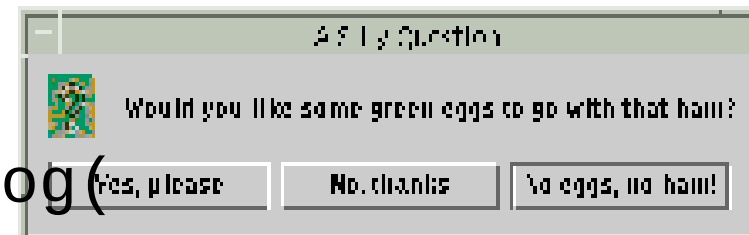
- Sunt create cu metode statice;
- nu sunt foarte orientate obiect
- Nu sunt foarte puternice





Tipuri de JOptionPane

- `public static void showMessageDialog(Component parent, Object message)`
- `public static int showConfirmDialog(Component parent, Object message)`
- `public static String showInputDialog(Component parent, Object message)`



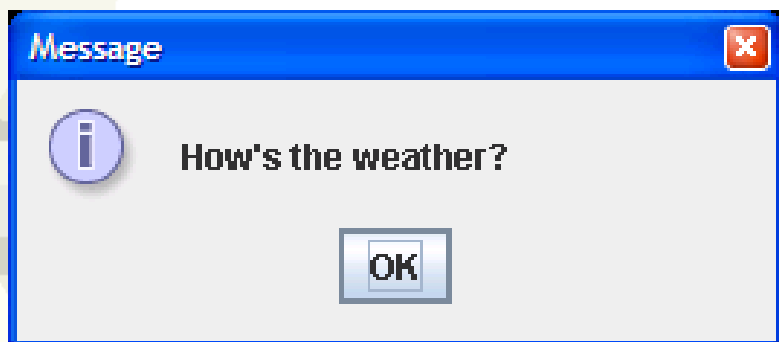


JOptionPane exemplul 1

- `showMessageDialog` este analog cu `System.out.println` pentru afișarea unui simplu mesaj

```
import javax.swing.*;
```

```
class MessageDialogExample {  
    public static void main(String[] args) {  
        JOptionPane.showMessageDialog(null,  
            "How's the weather?");  
        JOptionPane.showMessageDialog(null,  
            "Second message");  
    }  
}
```

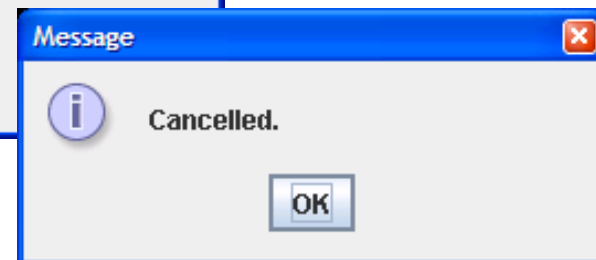
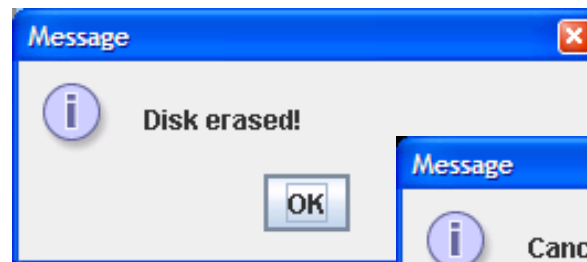
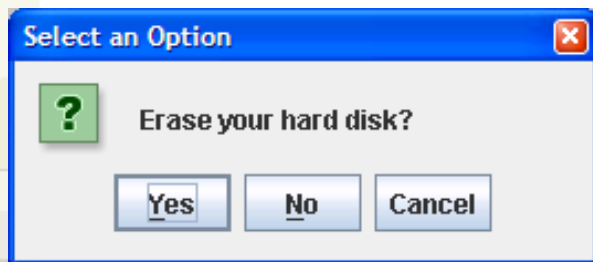




JOptionPane exemplul 2

```
import javax.swing.*;

class ConfirmDialogExample {
    public static void main(String[] args) {
        int choice = JOptionPane.showConfirmDialog(null,
            "Erase your hard disk?");
        if (choice == JOptionPane.YES_OPTION) {
            JOptionPane.showMessageDialog(null, "Disk erased!");
        } else {
            JOptionPane.showMessageDialog(null, "Cancelled.");
        }
    }
}
```

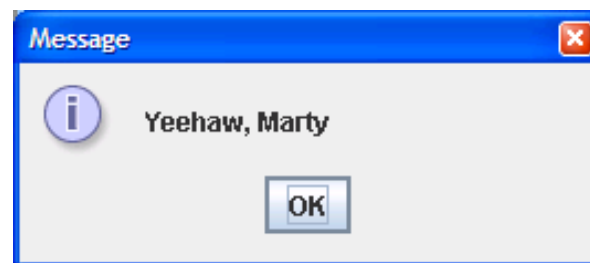
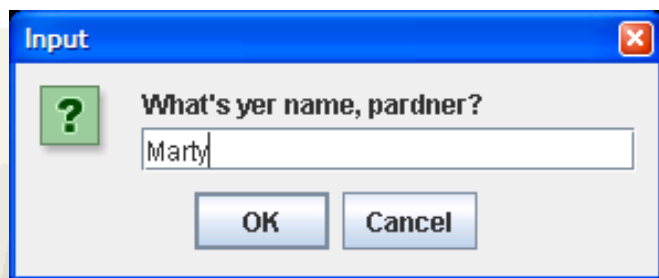




JOptionPane example 3

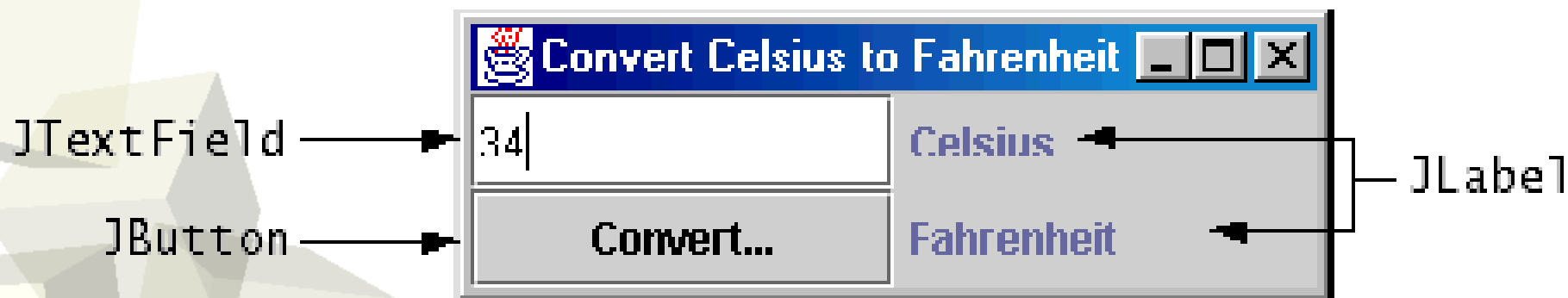
```
import javax.swing.*;

class InputDialogExample {
    public static void main(String[] args) {
        String name = JOptionPane.showInputDialog(null,
            "What's yer name, pardner?");
        JOptionPane.showMessageDialog(null, "Yeehaw, " + name);
    }
}
```





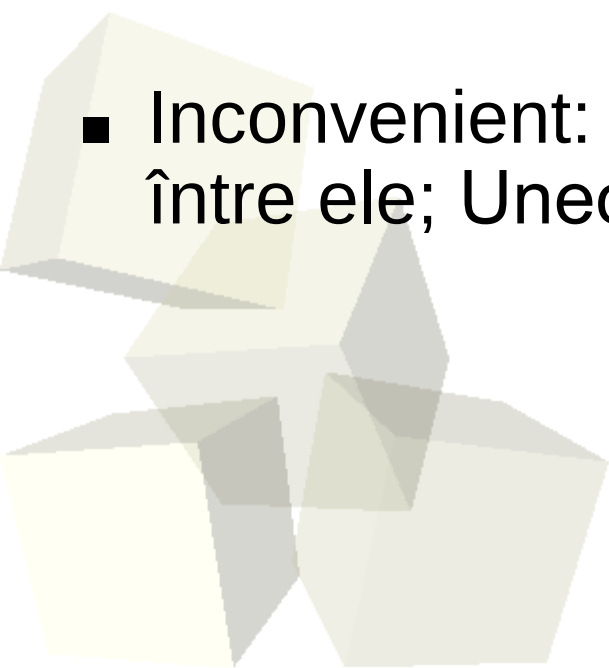
- **windows:** container de tip top-level
exemple: frame, dialog box
- **componente GUI:**
exemple: button, text box, label
- **containere:** grupuri logice de componente
- *exemple: panel*





Java GUI: AWT și Swing

- Ideea inițială a firmei Sun: crearea unui set de clase/metode care să se folosească pentru a implementa interfețe GUI multiplatformă (Abstract Windowing Toolkit, sau **AWT**)
 - problemă: nu sunt suficient de puternice; mai greoi de folosit
- A doua ediție (JDK v1.2): **Swing**
 - o bibliotecă nouă, care permite grafica mult mai performantă și construcții GUI
- Inconvenient: Există ambele în Java; ușor de încurcat între ele; Uneori este necesar să se utilizeze ambele!





- Swing nu inlocuieste modelul AWT ci il extinde
- Aplicatiile GUI vor avea in continuare nevoie de pachetul java.awt
- Exemplul 1 AWT

```
■ import java . awt . * ;  
■ import java . awt . event . * ;  
■ public class ExempluAWT extends Frame implements  
■ ActionListener {  
■ public ExempluAWT ( String titlu ) {  
■ super ( titlu );  
■ setLayout ( new FlowLayout ( ) );  
■ add ( new Label ( " Hello AWT " ) );  
■ Button b = new Button ( " Close " );  
■ b . addActionListener ( this );  
■ add ( b );  
■ pack ( ); show ( );  
■ }  
■ public void actionPerformed ( ActionEvent e ) {  
■ System . exit ( 0 ); }  
■ public static void main ( String args [ ] ) {  
■ new ExempluAWT ( " Hello " );  
■ } }
```



- Aplicatia rescrisa folosind Swing
- `import javax . swing . * ;`
- `import java . awt . * ;`
- `public class ExempluSwing extends JFrame implements`
- `ActionListener {`
- `public ExempluSwing ( String titlu ) {`
- `super ( titlu );`
- `// Metoda setLayout nu se aplica direct ferestrei`
- `getContentPane (). setLayout (new FlowLayout ());`
- `// Componentele au denumiri ce incep cu litera J`
- `// Textul poate fi si in format HTML`
- `getContentPane ().add( new JLabel (`
- `"<html ><u>Hello </u> <i>Swing </i ></ html >"));`
- `JButton b = new JButton (" Close ");`
- `b. addActionListener ( this );`
- `getContentPane ().add(b);`
- `pack ();show ();}`
- `public void actionPerformed ( ActionEvent e) {`
- `// Tratarea evenimentelor se face ca in AWT`
- `System . exit (0);`
- `}`
- `public static void main ( String args []) {`
- `new ExempluSwing (" Hello ");`
- `}`



Ierarhia componentelor Swing

```
java.lang.Object
+-- java.awt.Component
+-- java.awt.Container
|   +-- javax.swing.JComponent
|   |   +-- javax.swing.JButton
|   |   +-- javax.swing.JLabel
|   |   +-- javax.swing.JMenuBar
|   |   +-- javax.swing.JOptionPane
|   |   +-- javax.swing.JPanel
|   |   +-- javax.swing.JTextArea
|   |   +-- javax.swing.JTextField
|   +-- java.awt.Window
|       +-- java.awt.Frame
|           +-- javax.swing.JFrame
```

■ **import java.awt.*;**
import javax.swing.*;



- `public int getWidth()`
`public int getHeight()`
Returnează înălțimea și lățimea componentei în pixeli.
- `public boolean isEnabled()`
Returnează adev sau fals, după cum componenta poate sau nu să interacționeze.
- `public boolean isVisible()`
Returnează adev sau fals, după cum componenta este sau nu vizibilă pe ecran



Alte metode JComponent

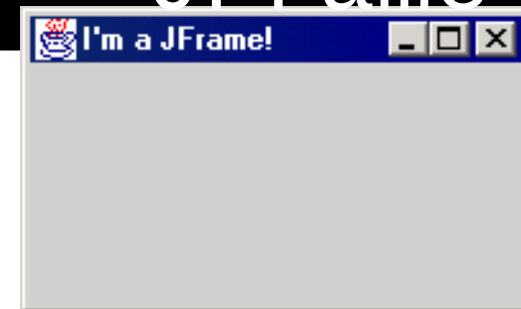
- `public void setBackground(Color c)`
Stabilește culoarea de fundal a componentei.
- `public void setFont(Font f)`
Stabilește fontul textului afișat pe componentă.
- `public void setEnabled(boolean b)`
Stabilește faptul că componenta este validă și interacționează cu user-ul
- `public void setVisible(boolean b)`
Stabilește dacă componenta va fi vizibilă sau nu.





Un frame este o componentă pe care se pot afișa și alte componente

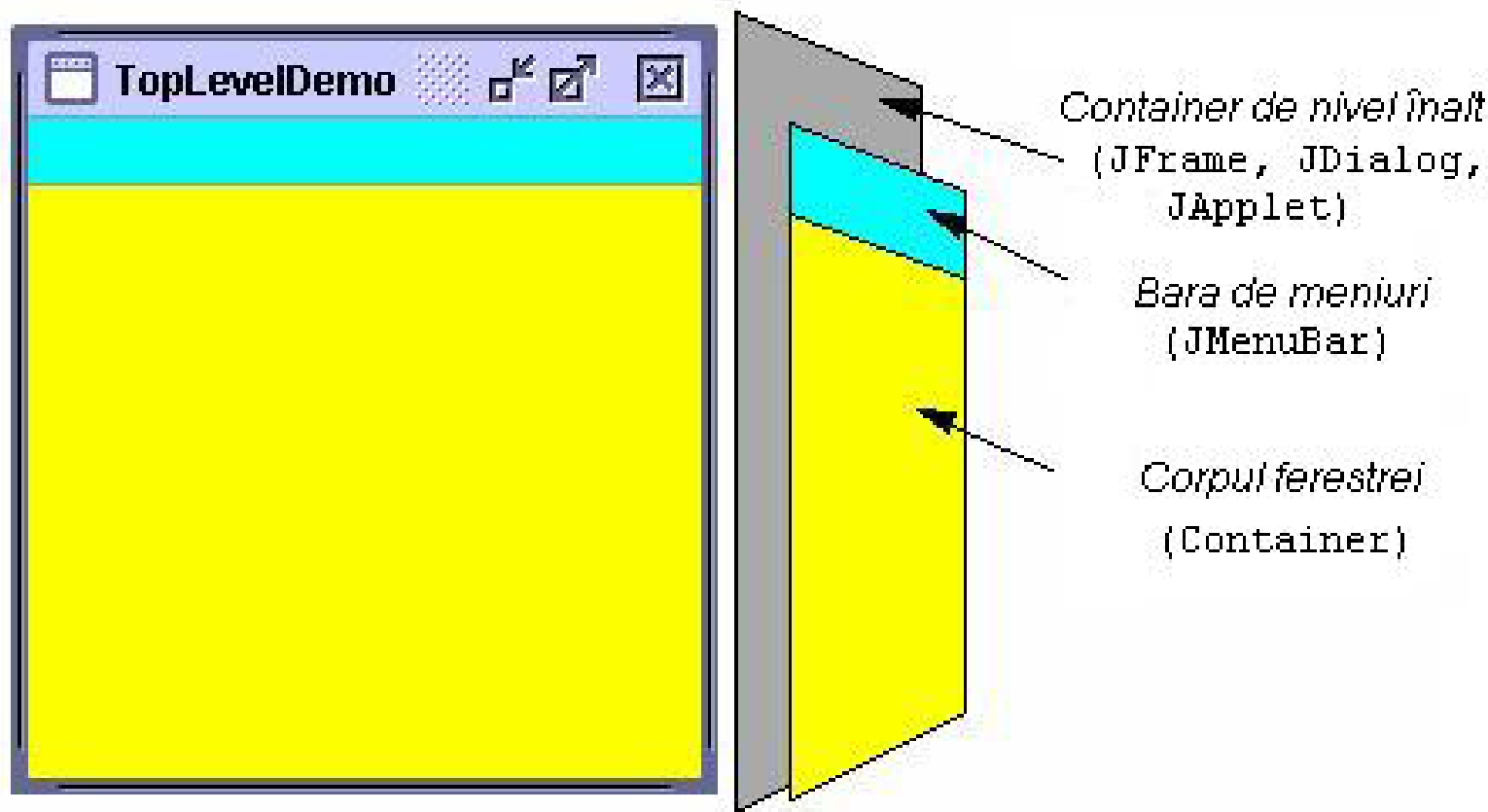
- `public JFrame()` sau `public JFrame(String title)`
Creează un frame cu un titlu opțional.
- `public void setTitle(String text)`
Afișează titlul în bara de titlu.
- `public void setDefaultCloseOperation(int op)`
Stabilește care va fi operația efectuată la închiderea ferestrei. Valoare implicită: `JFrame.EXIT_ON_CLOSE`
- `public void add(Component comp)`
Adaugă componenta dată sau containerul pe fereastra curentă
- `public void pack()`
Redimensionează la cea mai mică dimensiune.
- *Obs:* Se apelează `setVisible(true)` pentru a determina afișarea ferestrei după crearea ei.





Folosirea ferestrelor

- In general orice aplicatie Java independenta bazata pe Swing contine cel putin un container de nivel inalt reprezentat de fereastra principala a programului, instanta a clasei JFrame.



- Corpul ferestrei este o instanta a clasei Container ce poate fi obtinuta cu metoda `getContentPane`.



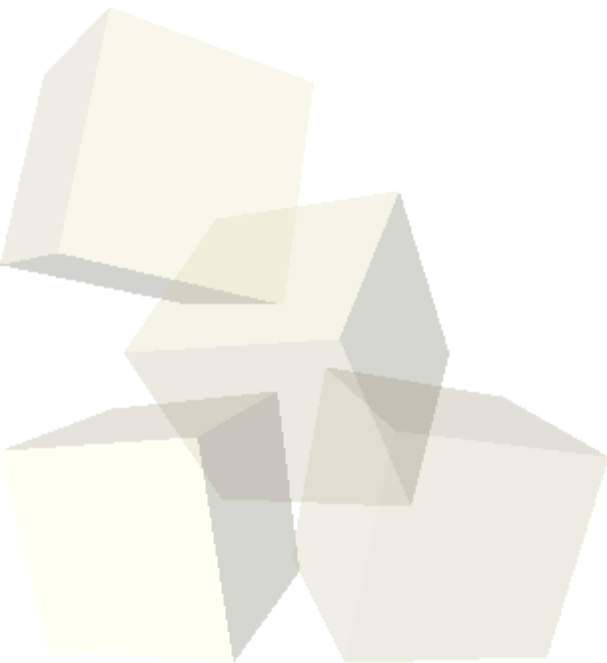
Folosirea ferestrelor Ferestre interne

- Din punctul de vedere al folosirii ferestrelor, aplicatiile pot fi impartite in
- doua categorii:
 - • SDI (Single Document Interface)
 - • MDI (Multiple Document Interface)
- In Swing, clasa JInternalFrame pune la dispozitie o modalitate de a crea ferestre in cadrul altor ferestre

```
■ import javax . swing . * ;  
■ import java . awt . * ;  
■ class FereastrPrincipala extends JFrame {  
■ public FereastrPrincipala ( String titlu ) {  
■ super ( titlu );  
■ setSize ( 300 , 200 ) ;  
■ setDefaultCloseOperation ( JFrame . EXIT_ON_CLOSE );  
■ FereastrInterna fin1 = new FereastrInterna ( );  
■ fin1 . setVisible ( true );  
■ FereastrInterna fin2 = new FereastrInterna ( );  
■ fin2 . setVisible ( true );
```



```
■ JDesktopPane desktop = new JDesktopPane ();
■ desktop .add( fin1 );
■ desktop .add( fin2 );
■ setContentPane ( desktop );
■ fin2 . moveToFront ();
■ } }
■ class FereastrăInterna extends JInternalFrame {
■ static int n = 0; // nr. de ferestre interne
■ static final int x = 30, y = 30;
■ public FereastrăInterna () {
■ super (" Document #" + (++ n),
■ true , // resizable
■ true , // closable
■ true , // maximizable
■ true );// iconifiable
■ setLocation (x*n, y*n);
■ setSize ( new Dimension (200 , 100) );
■ } }
■ public class TestInternalFrame {
■ public static void main ( String args []) {
■ new FereastrăPrincipala (" Test ferestre interne "). show
■ ();
■ } }
```



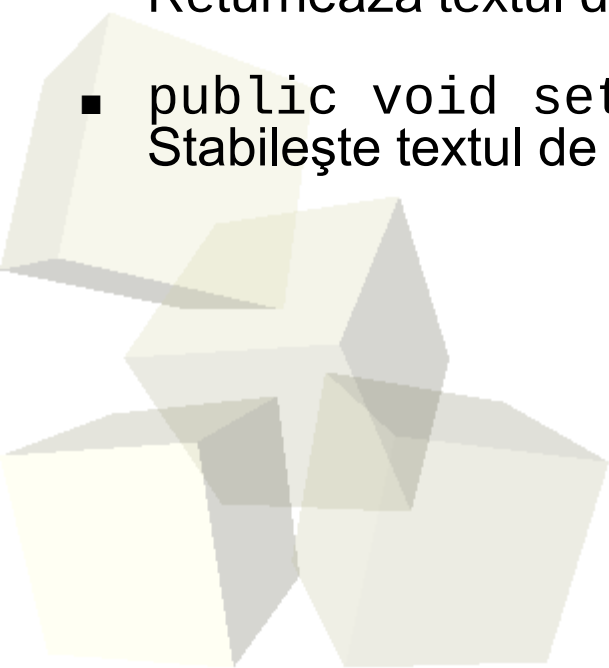


JButton, JLabel

*Cea mai comună componentă—
Regiune pe care se poate efectua
click și interacționează cu userul
pentru a efectua o singură comandă*

*Un text label afișează un text pe ecran
Nu interacționează cu user-ul*

- `public JButton(String text)`
`public JLabel(String text)`
- `public String getText()`
Returnează textul de pe buton / label.
- `public void setText(String text)`
Stabilește textul de pe buton / label





JTextField, JTextArea

Sunt componente de text editabil pe una sau pe mai multe linii

- `public JTextField(int columns)`
- `public JTextArea(int lines, int columns)`
-
- `public String getText()`
returnează textul.
- `public void setText(String text)`
stabilește textul.

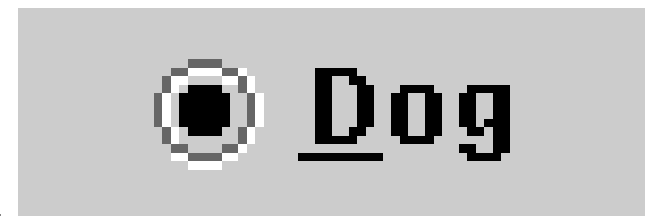
George Washington

Thomas Jefferson

Verify that the RJ45 cable is connected to the WAN plug on the back of the Pipeline unit.



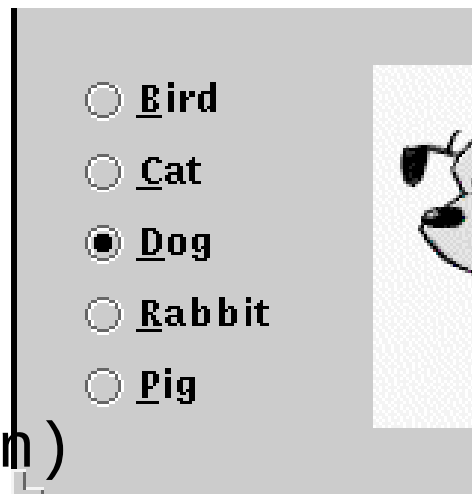
JCheckBox, JRadioButton



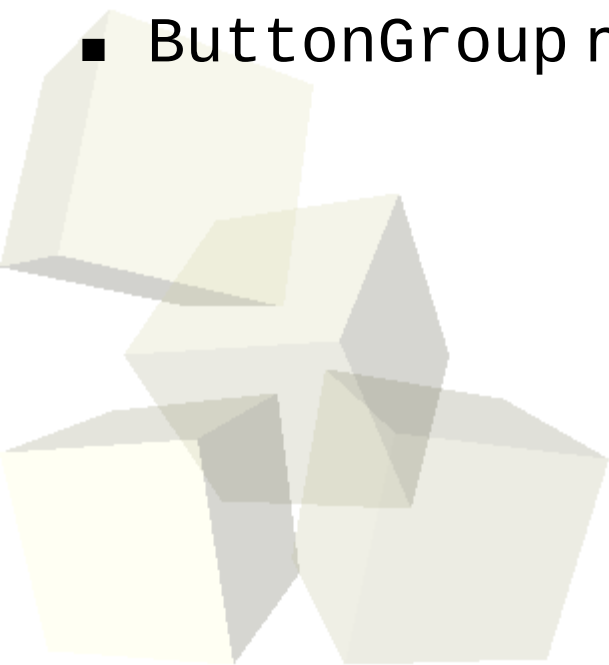
- `public JCheckBox / JRadioButton(String text)`
`public JCheckBox(String text, boolean isChecked)`
- `public boolean isSelected()`
- `public void setSelected(boolean selected)`



- `public ButtonGroup()`
- `public void add(JRadioButton button)`



- ButtonGroup nu este o componentă grafică





Permite adăugarea unei imagini la un buton, label sau altă componentă grafică

- `public class ImageIcon implements Icon`
 - `public ImageIcon(String filename)`
 - `public ImageIcon(URL address)`

- in JButton, JRadioButton, JCheckBox, JLabel, etc...
 - Metode ce utilizează Icon
 - `public void setIcon(Icon)`
 - `public void setSelectedIcon(Icon)`
 - `public void setRolloverIcon(Icon)`

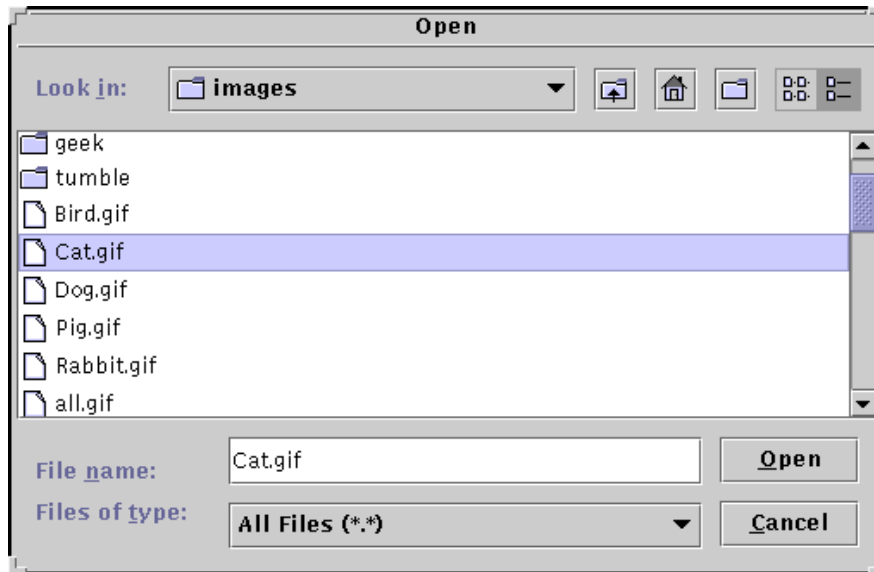


Container special care utilizează bare de derulare pentru a afișa componente

- `public JScrollPane(Component comp)`

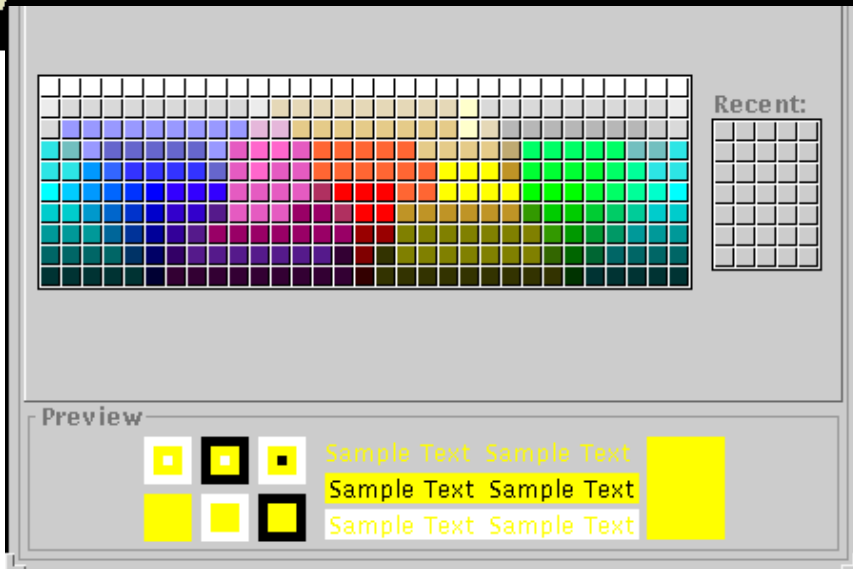
Se adaugă apoi pe suprafața containerului

```
contentPane.add(new JScrollPane(textarea),  
                BorderLayout.CENTER);
```



- `public JFileChooser()`
- `public JFileChooser(String currentDir)`
- `public int showOpenDialog(Component parent)`
- `public int showSaveDialog(Component parent)`
- `public File getSelectedFile()`
- `public static int APPROVE_OPTION, CANCEL_OPTION`
Rezultate posibile de la `showXxxDialog(..)`.

```
JFileChooser chooser = new JFileChooser();
int result = chooser.showSaveDialog(this);
if (result == JFileChooser.APPROVE_OPTION)
    this.saveData(chooser.getSelectedFile().getName());
```



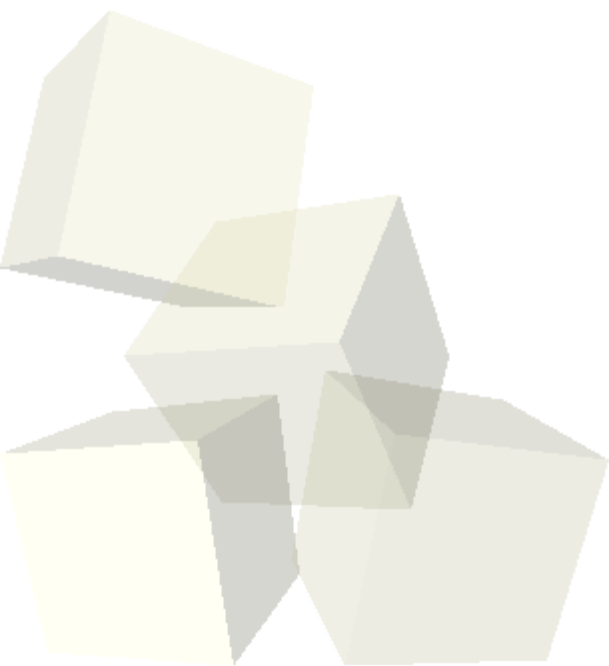
- `public JColorChooser()`
- `public JColorChooser(Color initial)`
- `public Color showDialog(Component parent, String title, Color initialColor)`
 - returnează null dacă utilizatorul selectează opțiunea Cancel

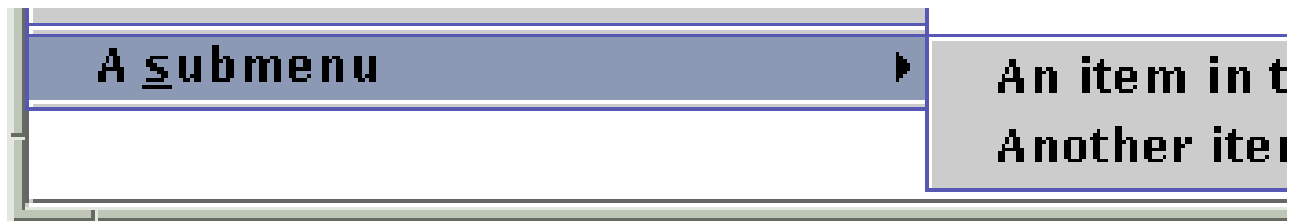


- `public JMenuBar()`
- `public void add(JMenu menu)`

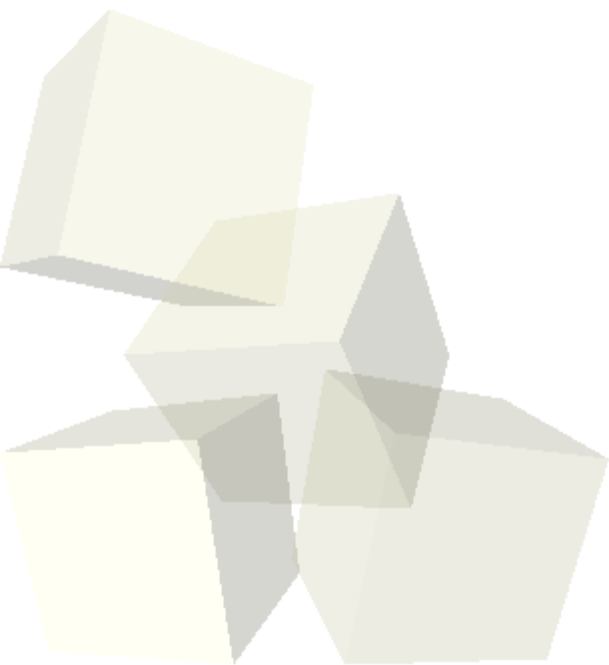
Mod de utilizare: in JFrame:

- `public void setJMenuBar(JMenuBar bar)`





- `public JMenu(String text)`
- `public void add(JMenuItem item)`
- `public void addSeparator()`
- `public void setMnemonic(int mnemonic)`





A text-only menu item Alt+1

 Both text and icon

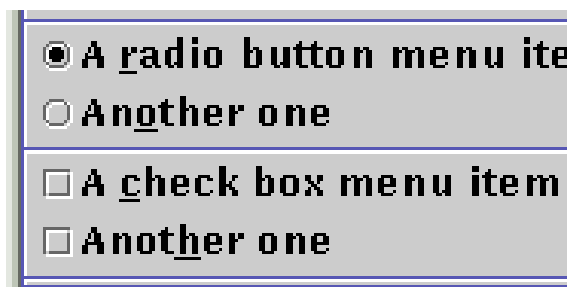
- `public JMenuItem(String text)`
- `public JMenuItem(String text, Icon icon)`
- `public JMenuItem(String text, int mnemonic)`

- `public void addActionListener(ActionListener al)`
- `public void setAccelerator(KeyStroke ks)`
- `public void setEnabled(boolean b)`
- `public void setMnemonic(int mnemonic)`



JCheckBoxMenuItem / JRadioButtonMenuItem

- `public J_____MenuItem(String text)`
- `public J_____MenuItem(String text, boolean select)`
- `public J_____MenuItem(String text, Icon icon)`
- `public J_____MenuItem(String text, Icon icon, boolean selected)`
- `public void addActionListener(ActionListener al)`
- `public boolean isSelected()`
- `public void setSelected(boolean b)`





- **mnemonic:** tasta “fierbinte” (hotkey) pt un button sau alt[comp grafică
- Usual este vizibilă fiind subliniată și se activează prin tastarea Ctrl+key (butoane) sau Alt+key (meniuri)
- Funcționează doar dacă componenta este în focar
- **utilizare:** apel metodă `setMnemonic(char)`

```
myQuitButton.setMnemonic('Q');
```

```
JMenuItem myNewItem = new JMenuItem("New", 'N');  
// sau,  
myNewItem.setMnemonic('N');
```



Both text and icon



- **acceleratoare:** combinație globală de taste care efectuează o acțiune (ex: Alt-X pt. exit din program) chiar dacă componenta nu este în focar
 - Poate rula în orice moment în cadrul unei aplicații
 - Opțional poate utiliza și modificatori (Shift, Alt)
 - Se creează prin apelul metodei `getKeyStroke` method din clasa `KeyStroke`, și transmiterea acesteia către metoda `setAccelerator` a diferitelor componente (buttons, menus)
- `menuItem.setAccelerator(
KeyStroke.getKeyStroke('T', KeyEvent.ALT_MASK));`

An item in the submenu

Alt+2



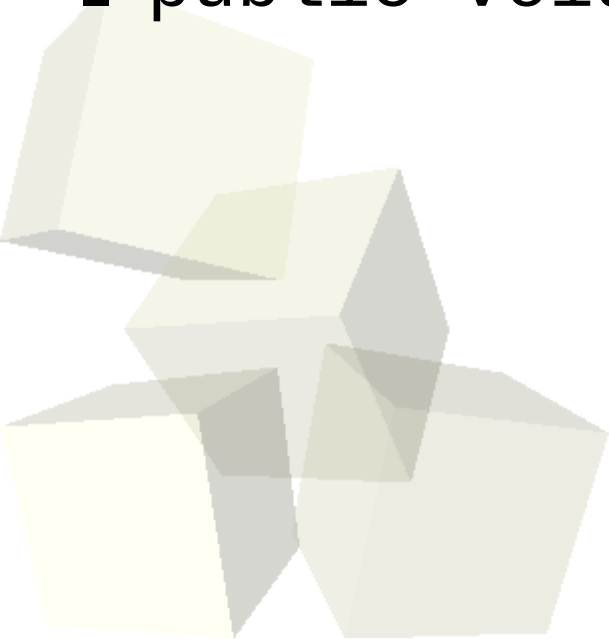
- `public JComboBox()`
- `public JComboBox(Vector items)`
- `public JComboBox(ComboBoxModel model)`
- `public void addItem(Object item)`
- `public Object getItemAt(int index)`
- `public void removeAllItems()`
- `public void removeItem(Object item)`
- `public void removeItemAt(int index)`





JComboBox: Selected Item

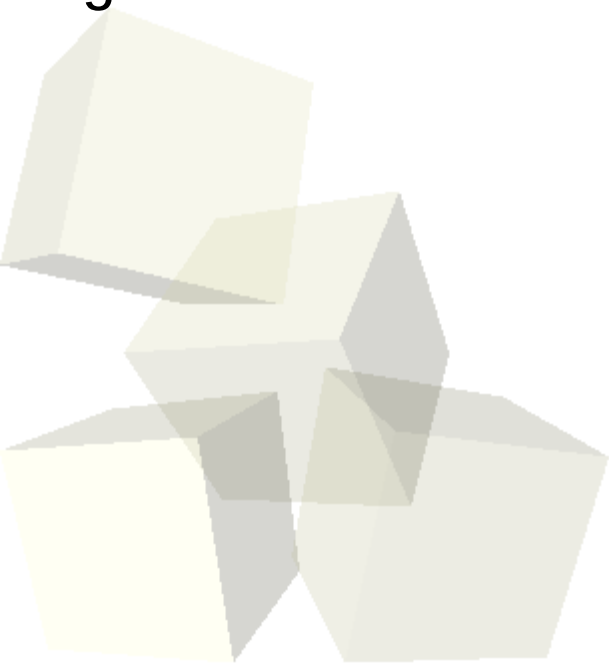
- `public int getSelectedIndex()`
- `public Object getSelectedItem()`
- `public void setSelectedItem(Object item)`
- `public void setSelectedIndex(int index)`
- `public void setEnabled(boolean enabled)`
- `public void setEditable(boolean editable)`





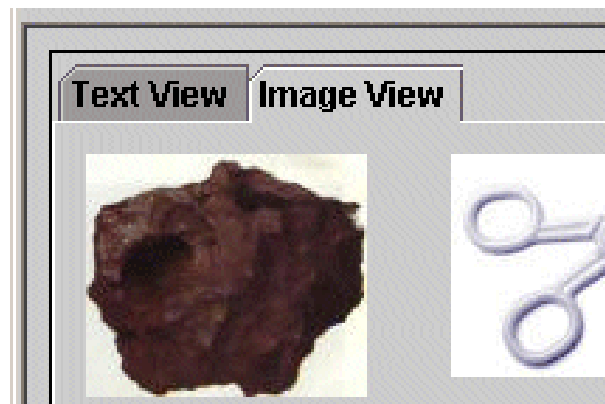
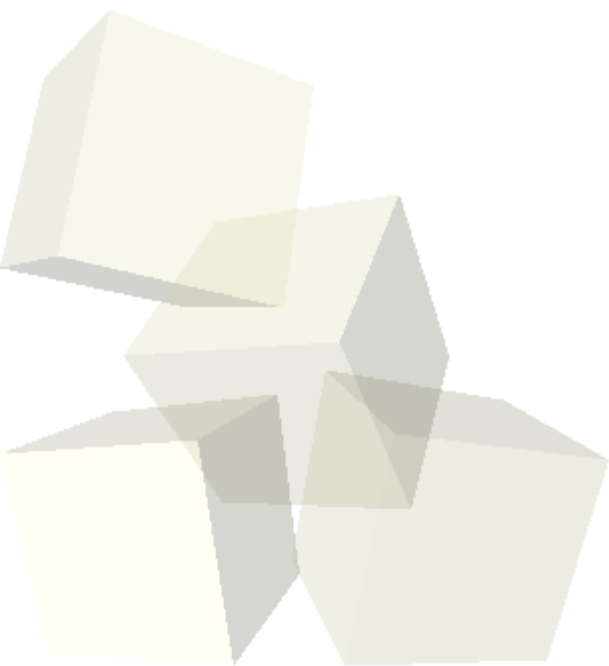
JComboBox Exemplu

```
final JComboBox box = new JComboBox();
box.addItem("Mirela");    box.addItem("Alin");
box.addItem("Jeni");
box.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent event) {
        JOptionPane.showMessageDialog(null,
            "You chose " + box.getSelectedItem());
    }
});
getContentPane().add(box, BorderLayout.NORTH);
```





- `public JTabbedPane()`
- `public JTabbedPane(int tabAlignment)`
Construiesc o nouă componentă tabbed pane. Implicit tab-urile sunt sus; se pot seta `JTabbedPane.BOTTOM`, `LEFT`, `RIGHT`, etc.
- `public void addTab(String title, Component comp)`
- `public void addTab(String title, Icon icon, Component comp)`
- `public void addTab(String title, Icon icon, Component comp, String tooltip)`





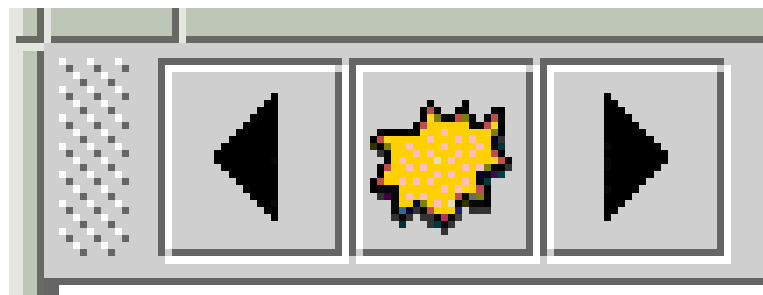
metode JTabbedPane

- `public void insertTab(String title, Icon icon, Component comp, String tooltip, int index)`
- `public void remove(Component comp)`
- `public void remove(int index)`
- `public void removeAll()`
- `public void setSelectedComponent(Component c)`
- `public void setSelectedIndex(int index)`



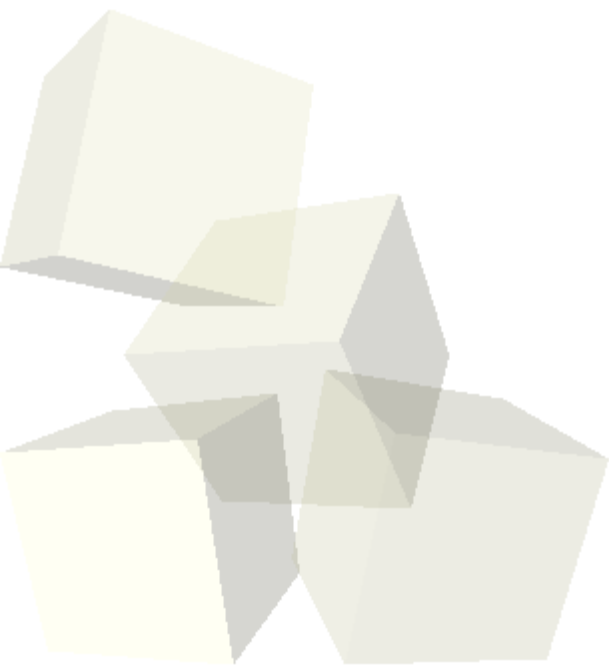


- `public JToolBar()`
- `public JToolBar(int orientation)`
- `public JToolBar(String title)`
- `public JToolBar(String title, int orientation)`
Construiesc un tool bar, optional cu un titlu și o orientare;
poate fi `JToolBar.HORIZONTAL` sau `VERTICAL`, implicit
horizontal
- `public void add(Component comp)`
adaugă o nouă componentă.





- Apel constructor toolbar
- add items (usual buttoane)
- add toolbar la content pane cu ajutorul BorderLayout (usual NORTH)



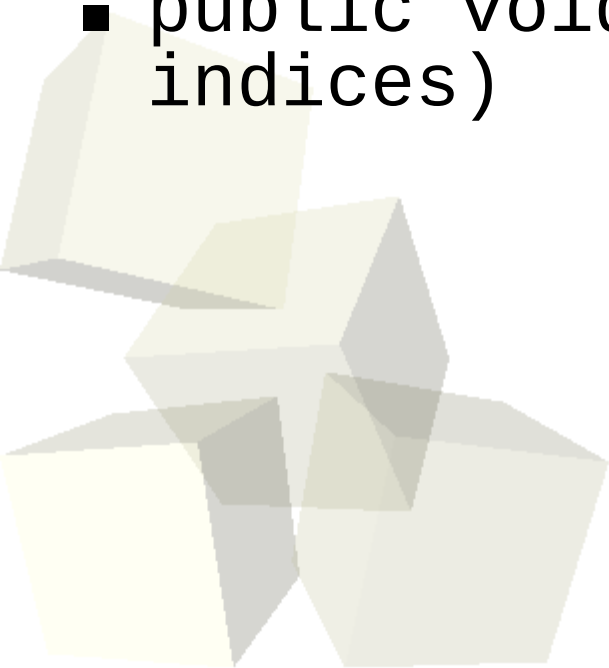


- `public JList()`
Construiește un JList gol.
- `public JList(ListModel model)`
- `public JList(Object[] data)`
- `public JList(Vector data)`
Constructs a JList care afișează datele primite.
- `public void addListSelectionListener(
 ListSelectionListener lsl)`
Pentru răspuns la evenimente legate de liste.



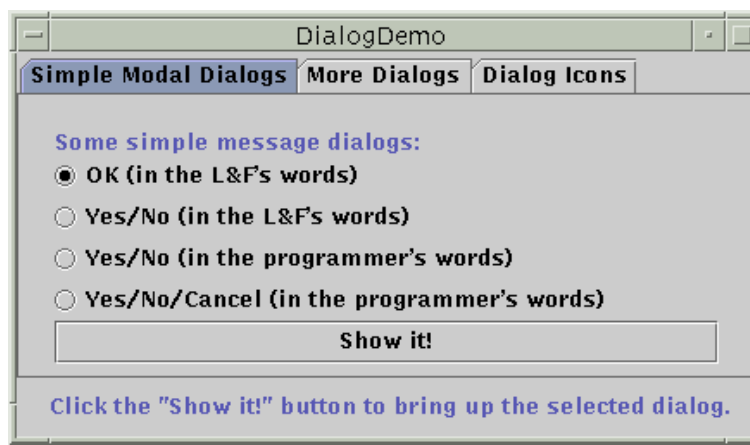


- `public void clearSelection()`
- `public int getSelectedIndex()`
- `public int[] getSelectedIndices()`
- `public Object getSelectedValue()`
- `public Object[] getSelectedValues()`
- `public void setSelectedIndex(int index)`
- `public void setSelectedIndices(int[] indices)`





- `public JDialog(Dialog parent, boolean modal)`
- `public JDialog(Frame parent, String title, boolean modal)`
- `public void show()`
- JDialog posedă cele mai multe metode JFrame :
`getContentPane()`, `setJMenuBar(JMenuBar)`,
`setResizable(boolean)`, `setTitle(String)`, ...





Layout Management

■ Exemple

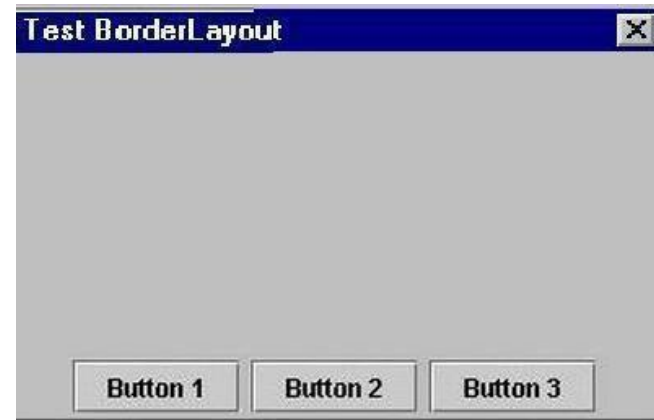
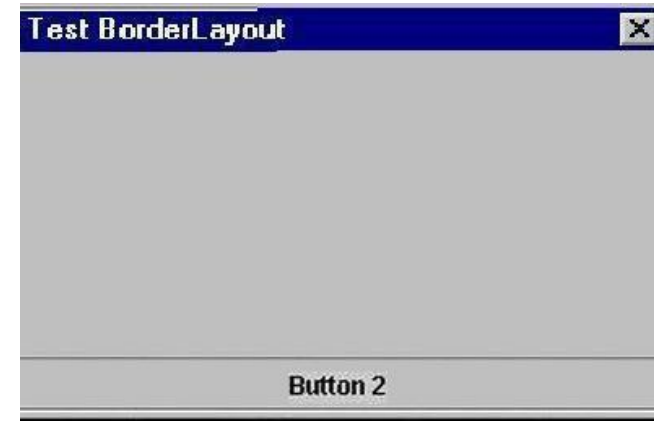


- Toate componentele dintr-un container sunt poziționate de un *layout manager*. În exemplul anterior s-a utilizat *flow layout manager*, care este managerul implicit de componente a unui panel.
 - The default manager lines the components horizontally until there is no more room and then start a new row of components
 - After resizing the container, the layout manager reflows the components automatically
 - The default is to center the components in each row. You can choose aligning them to the left or to the right of the container.
`panel.setLayout(new FlowLayout(FlowLayout.LEFT));`
- Other managers: see <http://java.sun.com/docs/books/tutorial/uiswing/layout/visual.html>.



Layout Management — Using Panel

- Potential problem with BorderLayout:
 - The button is stretched to fill the entire southern region of the frame
 - If you add another button to the southern region, it would just displace the first button
- Solution – use additional *panels*.
 - It acts as containers for interface elements and can themselves be arranged inside a larger panel
 - Use flow layout by default
 - To fix the problem of BorderLayout
 1. Create a panel
 2. Add components to the panel
 3. Add the panel to the larger container



```
JPanel p = new JPanel();  
p.add(button1);  
p.add(button2);  
p.add(button3);  
frame.add(panel, BorderLayout.SOUTH);
```



Containere cu layout

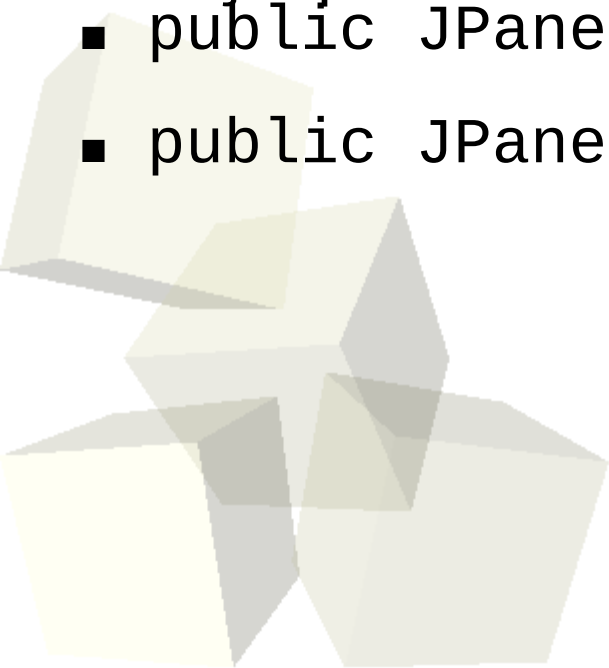
The image displays five Java Swing windows, each demonstrating a different layout manager. Each window has a title bar with a standard icon and window controls (minimize, maximize, close).

- BorderLayout:** The components are arranged in a border layout. 'Button 1' is at the top center. Below it, 'Button 3' and 'Button 5' are on the left and right respectively, with '2' in the center. 'Long-Named Button 4' is at the bottom center.
- GridLayout:** The components are arranged in a 3x2 grid. The top row contains 'Button 1' and '2'. The middle row contains 'Button 3' and 'Long-Named Button 4'. The bottom row contains 'Button 5' and an empty space.
- FlowLayout:** The components are arranged in a single row, wrapped as needed. From left to right: 'Button 1', '2', 'Button 3', 'Long-Named Button 4', and 'Button 5'.
- BoxLayout:** The components are arranged vertically in a single column. From top to bottom: 'Button 1', '2', 'Button 3', 'Long-Named Button 4', and 'Button 5'.
- GridBagLayout:** The components are arranged in a grid-like fashion. The top row contains 'Button 1', '2', and 'Button 3'. Below them is a large area containing 'Long-Named Button 4'. At the bottom right is 'Button 5'.



container: *un obiect care poate păstra componente;*

- `public void add(Component comp)`
`public void add(Component comp, Object info)`
- `public void remove(Component comp)`
- `public void setLayout(LayoutManager mgr)`
- `public void validate()`
Se apelează în caz de modificare conținut container care este deja afișat pe ecran.
- `public JPanel()`
- `public JPanel(LayoutManager mgr)`



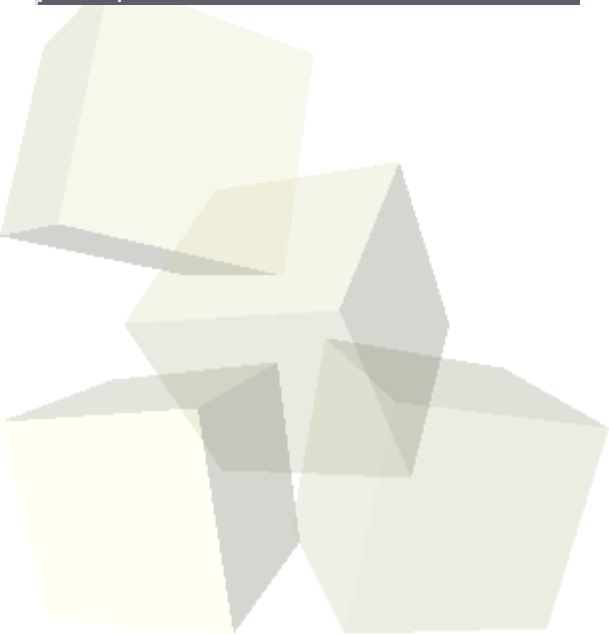
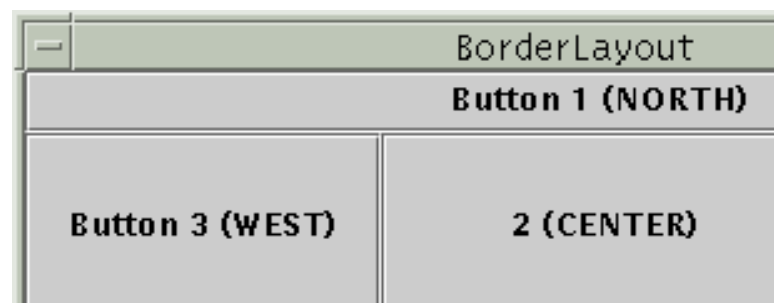


Dimensionare componente

*Buttons dimensionate
automată:
după preferință:*



Dimensionare

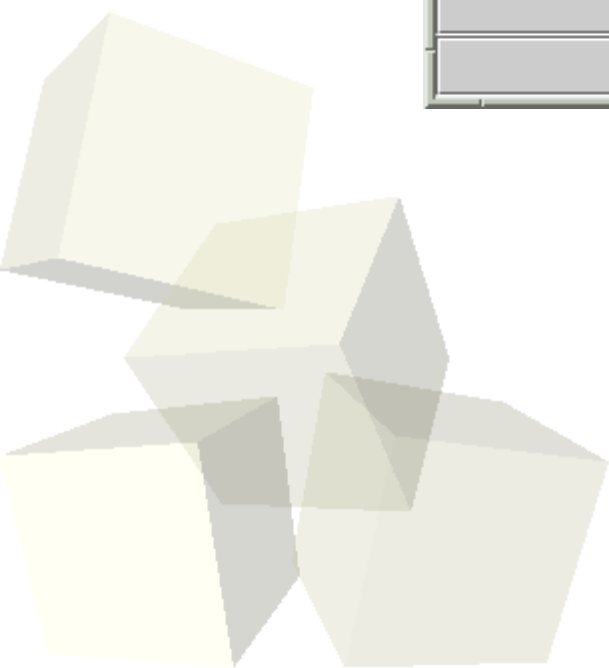




BorderLayout

```
public BorderLayout()
```

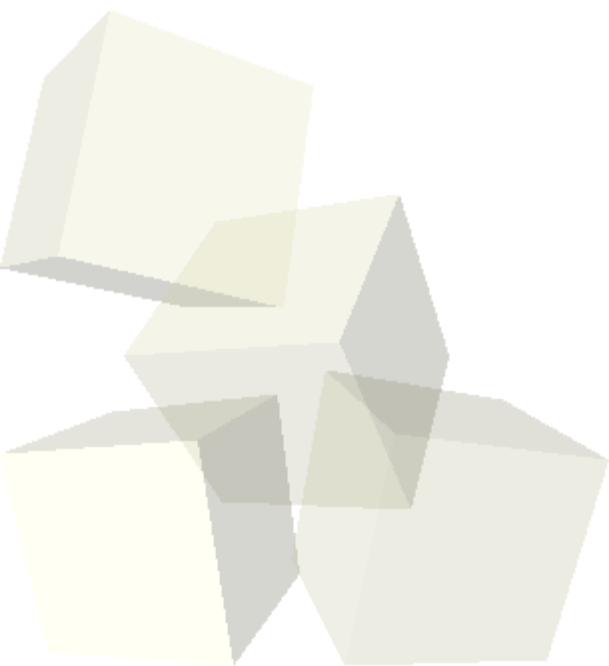
```
Container panel = new JPanel(new BorderLayout());  
panel.add(new JButton("Button 1 (NORTH)", BorderLayout.NORTH);
```





FlowLayout

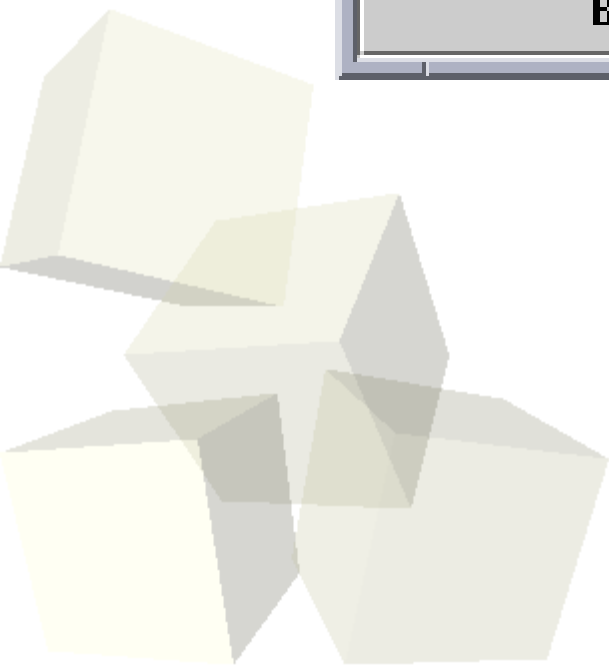
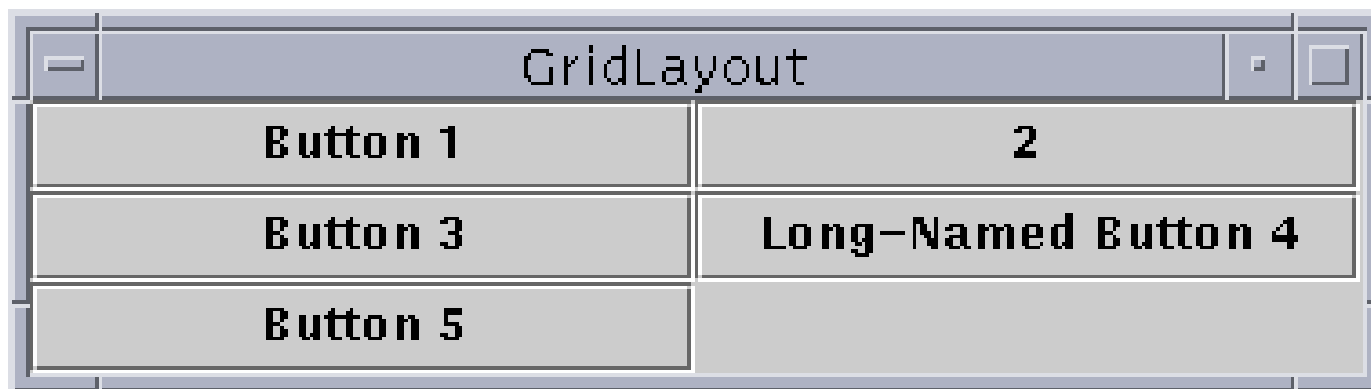
```
public FlowLayout()  
Container panel = new JPanel(new FlowLayout());  
panel.add(new JButton("Button 1"));
```





GridLayout

```
public GridLayout(int rows, int columns)
```

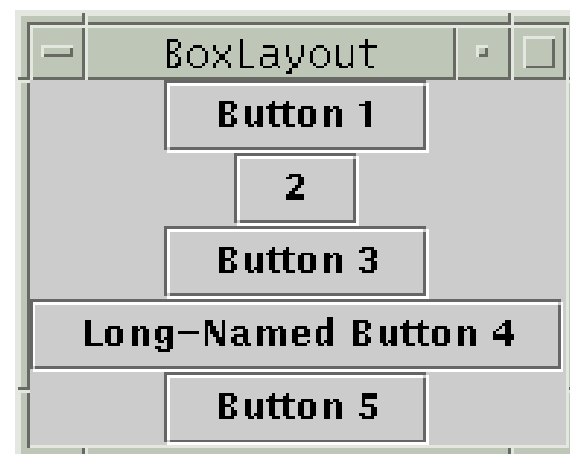
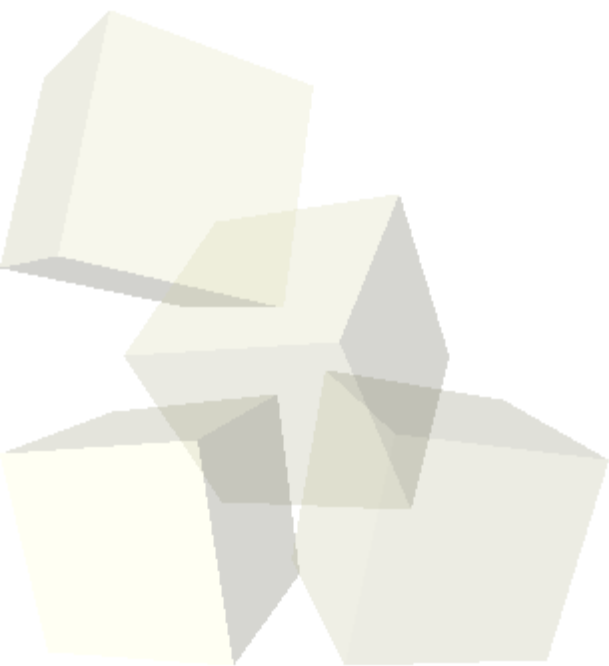




```
Box.createHorizontalBox()  
Box.createVerticalBox()
```



```
Box.createHorizontalBox(); sau  
Box.createVerticalBox();
```





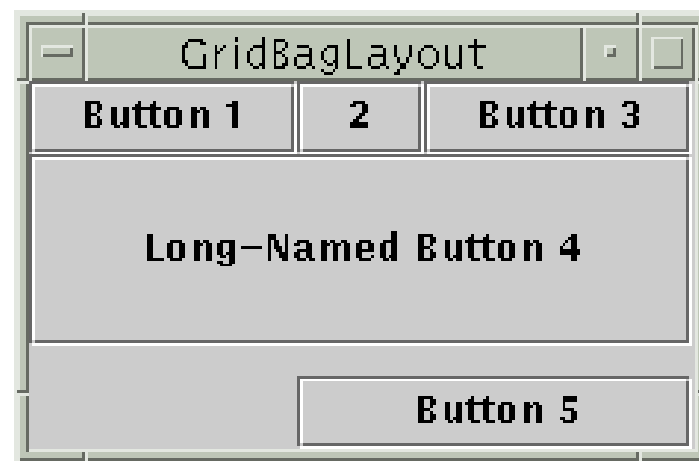
Alte tipuri de layout-uri

■ CardLayout



■ GridBagLayout

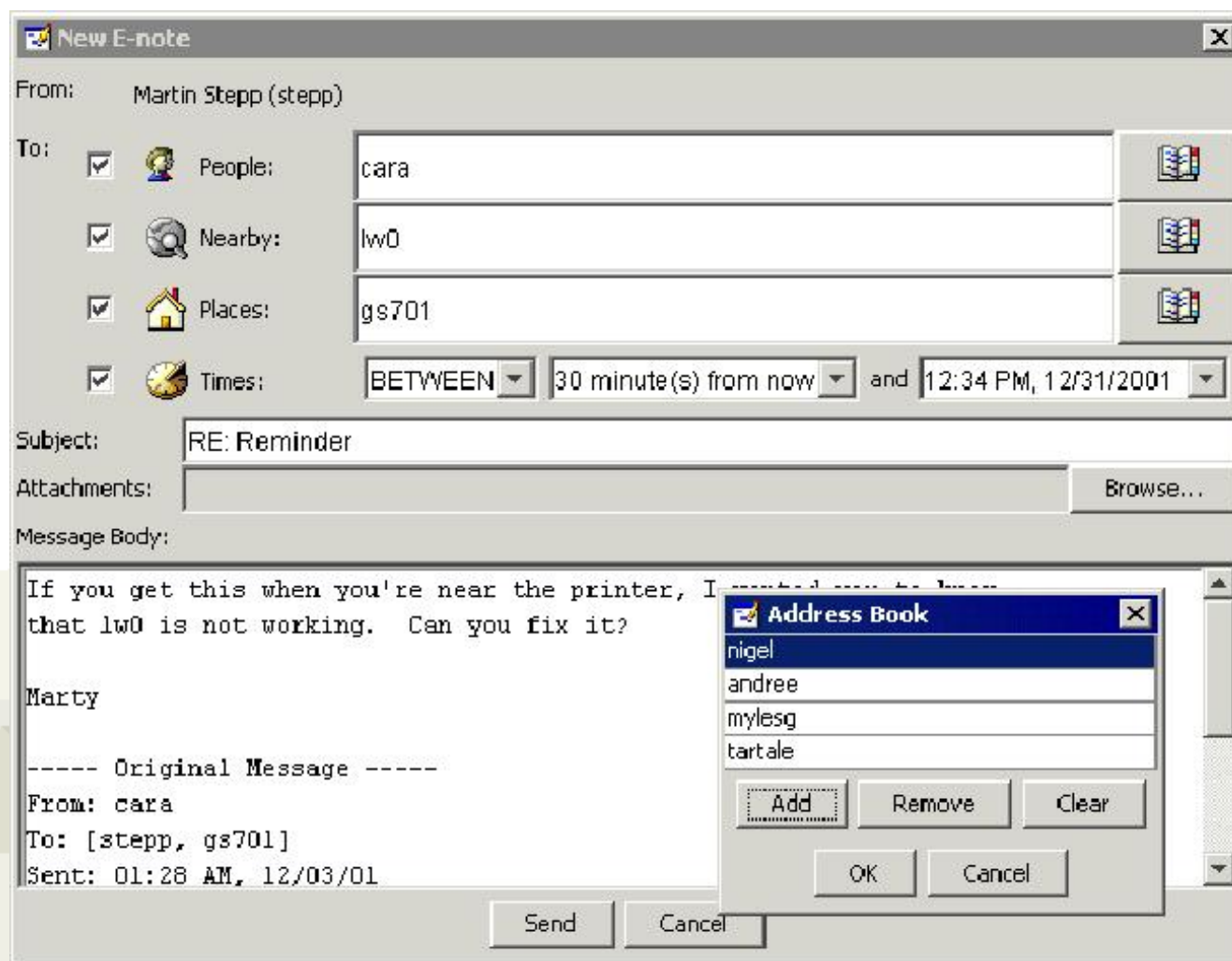
- custom / null layout
poziționare absolută cu `setX/Y` și
`setWidth/Height`





Probleme cu layout managerii

Cum se poate crea o GUI complexă, ca cea din imagine?





Soluție: combinarea layout-urilor

- Se creează panouri intermediare
- Fiecare panou are alt layout,

